

swtpcsim — User Manual

2026-09-27 · 0e4d89e

*A simulation of the SWTPC 6800 on the SS-50/SS-30 bus, and the MITS Altair
680b*

What swtpcsim is	1
What is in the package	3
Running it	7
Quick start: boot a machine in one command	9
Quick reference	14
Machines	22
The machine file	29
Boards	43
Disks	51
Tapes	58
Serial ports, sockets and telnet	66
Worked examples	81
The MCP server	86
Troubleshooting	93
Glossary	98
Boards and their parameters	101
The built-in machines	110

What swtpcsim is

`swtpcsim` simulates the **SWTPC 6800** and the **MITs Altair 680b** — the Motorola 6800 machines of the mid-1970s and the buses they were built around.

It boots real software — not software written to work with it, but the actual artifacts, byte for byte, as they were shipped: SWTBUG and MIKBUG in ROM, TSC's FLEX off a floppy, MON680 on the 680b. None of it has been patched, and none of it knows it is not running on a real machine.

The bus is a real object in the program, not a wiring diagram implied by the code. Boards plug into it, answer addresses, pull the interrupt line, and float the data bus when nobody is driving it — getting the answer wrong in exactly the ways real boards did. That makes the machine a **bench**: a board you have not built yet can be fitted here, and the driver you have not finished can be run against it, before either exists in copper. And it makes bugs findable — you can stop the machine mid-instruction, ask which board answered and which stayed silent, and run the same thing again and get the same answer, because nothing here is intermittent and nothing is hidden.

What it does

- **A Motorola 6800**, faithful down to the flags and the cycle counts, and checked against a processor reference before any board is built on it. It drives two machines: the SWTPC 6800 on its SS-50/SS-30 bus, and the MITs Altair 680b.
- **A board for most of the machine**, each modelled from its own manual: the CPU board, RAM/ROM, the SWTPC MP-S serial console, the Altair 680b's onboard and Universal I/O, the KCACR audio-cassette interface, and the DC-4 floppy controller that FLEX boots from. The boards chapter has the whole list.
- **A monitor** — the prompt you get when the machine is not running — with breakpoints (plain or conditional), single-stepping, disassembly, memory examine and deposit, a bus-cycle trace and a history ring, and a view of the bus itself: who decodes what, who is pulling the interrupt line, and where two boards are fighting.
- **Real I/O**. A serial board can be wired to your terminal, to a TCP socket (so you can telnet into the guest), or to an actual serial port on your machine, with the modem control lines wired through.

What it does not do

This section is here because a manual that only lists strengths is an advertisement.

- **You can save state, but not replay**. `SNAPSHOT` writes the machine's whole state to a file and `RESTORE` reads it back. What you cannot do is *record* a session and step backwards through it — there is no rewind.

- **These machines had no video and no audio.** A terminal on a serial port is the display, as it was. **There is no audio output.** Cassette `.WAV` files are read and written as *files*, which the tapes chapter covers.
- **Not every 6800 board is here**, and timing, while honest, is not a circuit simulation: instructions cost the right number of cycles and a cassette takes the right number of them to load, but propagation delays and analogue behaviour are not modelled — and no software from the period could tell.

What is in the box

You have the `swtpcsim` program, this manual, and a folder of worked examples with their media — complete machines that boot, each with a README of its own. More machines are built into the program itself. It boots something the moment you unzip it; the next chapter says what, and where further disks and tapes come from.

There is no installer and nothing to set up. Unzip it and run it.

What is in the package

swtpcsim	the program. One file, nothing to install.
QUICK-START.pdf	boot a machine in one command. Start here.
swtpcsim-manual.pdf	this.
swtpcsim-changelog.pdf	what changed in this release, and the ones before it.
swtpcsim-cheatsheet.pdf	every command and option, rendered for reading.
swtpcsim-monitor.pdf	the swtpcsim> prompt: driving the machine from the console.
swtpcsim-debugger.pdf	breakpoints, stepping, and looking at the bus itself.
DRIVING-WITH-AI.md	for an AI assistant driving the machine; see below.
cheatsheet.md	the same reference as plain text, for the AI to read.
LICENSE	the MIT licence this is published under.
LICENSE-SDL3	the licence of SDL3, which is built into the program.
examples/	machines that boot, media included.

That is the whole archive. There is no library to install, no runtime, and no configuration file you must write before the program will start.

Start with QUICK-START.pdf. It boots a machine in a single command and shows you the three keys — `^E`, `RUN`, `QUIT` — that move you between the CPU and the monitor. This manual is the long version of it; that one page is enough to get a machine running.

`swtpcsim-changelog.pdf` is the release history — what this version does that the last one did not, and the same for the versions before it. It is a separate document from this manual on purpose: the manual describes the program as it is *now*, and a record of what changed reads better on its own than as a chapter that would have to grow one section per release.

`swtpcsim-monitor.pdf` and `swtpcsim-debugger.pdf` are two more documents beside this one. They are about driving the program itself — the `swtpcsim>` prompt where you start and stop the machine, and the debugger you reach for at that prompt when something has gone wrong — rather than the emulated hardware this manual describes. Open them the way you open this one; each says at the top where to start if it is the first one you picked up.

`swtpcsim` is a single self-contained program. The one outside library it uses — **SDL3**, which opens the window the video boards draw into — is compiled *into* it rather than shipped beside it, so there is nothing to install and nothing that can go missing. `LICENSE-SDL3` is that library's licence, and it is in the package because its code is in the program.

The **Developer Guide** is not in here — it is a separate download from the same release page this came from, and you want it only if you intend to build a board of your own.

DRIVING-WITH-AI.md

This one is not for you, exactly. It is a briefing document for an **AI assistant**: drop it in a working directory, start an assistant there, and say *"using swtpcsim, boot FLEX and show me what is on the disk."* It tells the assistant how to drive the machine over the program's MCP interface. Ignore it if that is not how you work — nothing else depends on it.

The quick reference travels beside it: the whole command surface — every option, every monitor command, every board and machine — generated from this very program so it matches the binary you have. It ships in two forms of the same content. `swtpcsim-cheatsheet.pdf` is the one for you — open it the way you open this manual. `cheatsheet.md` is the same thing as plain text, there for the assistant to read.

The machines are in the program

You do not need any files to get a running machine — the machine descriptions are compiled into the binary, and naming one boots it:

```
$ swtpcsim --list          what the built-in names are
$ swtpcsim altair680       a MITS Altair 680b, MON680 in ROM, on a terminal
$ swtpcsim swtpc           a SWTPC 6800, SWTBUG in ROM, ready for a FLEX disk
```

A built-in is an ordinary machine file that happens to live inside the executable — the same TOML format you would write yourself, and `CONFIG SAVE mine.toml` writes any running machine out as one you can edit. `altair680` carries its monitor in ROM and needs nothing else; `swtpc` carries SWTBUG in ROM and comes up ready for a floppy, wanting media — which the next section is about. The machines chapter has the full story.

The examples, media included

`examples/` holds complete machines. Each is a folder with the media in it, so every one of them comes up the moment you unzip the archive — nothing to fetch, nothing to mount.

Every folder carries its own README, in Markdown and as a PDF beside it, and that README is the description of that example: what the machine is, what is in the drive or the deck, what to type, and what it should print back. So the list of examples is not written down in this manual — look in `examples/`, and read the README of whichever one you want.

```
$ ls examples/
$ swtpcsim examples/flex/flex2-40.toml
```

The folder is the unit, and you may move it anywhere. A path written *inside* a machine file resolves against **that file**, not against wherever you were standing when you ran the program — so `examples/flex/flex2-40.toml` names its disk as plain `FLEX2-40.DSK`, the one lying next to it, and the folder still boots after you copy it to your desktop, rename it, or mail it to somebody.

(The other half of that rule matters just as much: a path *you type* at the prompt is relative to **your shell**, because you are the one who can see your own directory. The machines chapter covers both halves.)

The examples chapter walks through some of them at length. Where an example carries period documentation of its own — a game's own printed manual, say — that travels in the folder too, and its README says so.

What is *not* in the package: everything else to run

What is in `examples/` is the whole of the shipped media. A built-in that wants a disk or a tape — `swtpc`, say, which comes up with an empty DC-4 drive — starts up perfectly well without one:

```
$ swtpcsim -x "SHOW MOUNTS" swtpc
swtpcsim> SHOW MOUNTS
UNIT          KIND  HOLDS
dc40:drive0   disk  (empty)

Paths are AS WRITTEN.  SHOW PATHS says what they are relative to.
```

You supply the media and **MOUNT** it. The disks and tapes chapters describe how — and where those chapters name an image that is not in `examples/`, they are showing you the shape of the command, not a file you already have.

Where the rest will come from. A separate `swtpcsim-packages` repository is planned to hold the wider collection of disks, tapes and machine files, packaged the same way — each example a self-contained folder you can drop anywhere. **It is not published yet**, and exactly which images go in it has not been settled, so there is nothing to link to here yet.

The bulk of the media is kept out of the program's own archive on purpose: an image is large, most of the good ones are not ours to redistribute, and the simulator's version and the software's have no reason to move together. The ones that ship are the ones that make the manual's first chapters true.

What is *not* in the package: the source

The **source code** is not here — the simulator's, that is. `swtpcsim` is an open project under the MIT licence, and the source is a separate thing to fetch:

<https://github.com/deltecent/swtpcsim>

Nothing in this manual requires it. The one exception worth naming: **if you want to build a board of your own** — which is what the simulator is really for — you need the source, and you want the *Developer Guide*, which is a different document. This one is about driving the machine, not extending it.

Reporting a bug, or asking for something

Both go in the same place — the **Issues** tab of that repository:

<https://github.com/deltecent/swtpcsim/issues>

Search it first; if nobody has raised your problem, open a new issue. You need a GitHub account, and nothing else.

What makes a bug report useful is enough for somebody else to see what you saw:

- The **version** — the line `swtpcsim` prints at startup, or `swtpcsim --version` — and which operating system. Paste it whole: the commit in the parentheses is what says which source built your copy, and between releases the number alone names them all the same. `SHOW VERSION` prints that from inside the monitor, plus a `video` row saying whether this copy can open a window — worth including in anything about the video boards.
- The **machine**: the built-in's name, or the machine file itself, which is a small text file you can paste.
- **What you typed and what happened**. Paste the terminal, prompt and all. The monitor echoes every command, so a pasted session is a complete record of what was asked of it.
- What you expected instead, when that is not obvious.

If the guest software misbehaved rather than the simulator, say which software and where you got it — a period program failing on real hardware in 1976 is a fair thing for it to do here too, and knowing the image is how that gets untangled.

A feature request is an issue as well, and does not need an apology. Say what you are trying to do rather than only which knob you want, because the machine often has a way in already; and if it does not, the shape of the problem is what decides the shape of the answer. A missing board is a particularly good request when you can name the manual it was documented in — every board here was modelled from its own documentation, and a board with no surviving source is one nobody can build honestly.

Running it

`swtpcsim` is a single executable. Unzip the package and run it from a terminal.

```
$ ./swtpcsim
swtpcsim 1.0.0 -- 6800, full speed.
machine: swtpc.  HELP for commands.
swtpcsim>
```

The `./` says **run the program in this directory** — on macOS and Linux a bare name is searched for on your `PATH` and not in the current directory, so the prefix is how the shell finds it. On Windows use `.\swtpcsim` in PowerShell (`cmd` finds it either way). Omit the prefix entirely once `swtpcsim` is on your `PATH` — then its bare name works from anywhere.

A release built straight from its tag prints just the version, as above. A build made *between* releases appends **the commit it was built from** — `swtpcsim 1.0.0-12-gabc1234` — because the version number alone names every build after a release alike, and it is the commit that says which source produced the program in front of you. Quote whichever you have in a bug report. `SHOW VERSION` prints the full identity on its own, and says whether the tree had uncommitted edits in it at the time.

That prompt is **the monitor**. The machine exists — it has memory, a processor, a console board and a floppy controller in it — but it is not running. Nothing has been started. This is the equivalent of standing in front of the real machine with the power on, before you have hit RESET and let its ROM monitor take over.

macOS: the first run

macOS marks anything that arrives from the internet and refuses to run it until you say otherwise. If you see *"cannot be opened because the developer cannot be verified"*, clear the mark:

```
$ xattr -dr com.apple.quarantine ./swtpcsim
```

Do that once. It is not a comment on the program; it is what macOS does to every unsigned binary that arrives in a zip, whoever wrote it.

The mark is put there by whatever *fetched* the file — a browser, a mail client — so if you pulled the zip down with `curl` or `scp` there may be nothing to clear. The command says nothing and succeeds either way, which is why it is `-dr` and not `-d`: plain `-d` reports an error when the flag is already absent, and that error is not a problem.

Getting help, and getting out

Type	To
HELP	list every command
HELP DUMP	the usage and worked examples for one command
QUIT	leave

There is no **EXIT**. **QUIT** is the word, and **Q** is enough of it.

Commands resolve by **prefix**, so you type as much as it takes to be unambiguous and no more — **HELP** shows each command with its shortest form in brackets: **D[UMP]** , **DE[POSIT]** , **RES[ET]** . Type the part before the bracket. And **commands are not case-sensitive**, nor are the names of the boards in the machine; this manual writes them in capitals only because it is easier to read.

Which machine you get

Running **swtpcsim** with no arguments gives you the default machine, **swtpc** — a SWTPC 6800 with a console and a DC-4 floppy controller, ready for a FLEX disk.

Naming something gets you something else:

```
$ ./swtpcsim examples/flex/flex2-40.toml  a machine file: this one boots FLEX
$ ./swtpcsim altair680                    a BUILT-IN machine, by name
$ ./swtpcsim --list                       what the built-in names are
```

A **built-in** is a machine file that lives inside the program. There is nothing special about it — it is written in the same format as the ones under **examples/** . To see what is actually in one, boot it and look:

```
$ ./swtpcsim -x BOARDS altair680
```

...and if you want it as a file you can edit, **CONFIG SAVE mine.toml** writes out the machine you are actually running, and what it writes will boot.

The full story — every command-line option, and how **swtpcsim** decides whether a word is a built-in name or a filename — is in the machines chapter.

Quick start: boot a machine in one command

```
$ ./swtpcsim swtpc
```

On **Windows** the program is `swtpcsim.exe` :

```
> swtpcsim.exe swtpc
```

That is the whole of it.

```
startup> RESET
RESET* pulsed. (Memory is UNTOUCHED -- only POWER loses RAM.)
startup> RUN
[console -- ^E returns to the monitor]

$
```

You are at **SWTBUG**, the SWTPC 6800's ROM monitor. The `$` is its prompt, and it is 1977. The bare `swtpc` machine comes up here with an empty floppy drive — from SWTBUG you examine memory, load a tape, or boot a disk.

Boot FLEX

To bring up an operating system, give the machine a disk and tell SWTBUG to boot it. The shipped FLEX example is that same machine with a FLEX 2.0 disk already in drive 0:

```
$ ./swtpcsim examples/flex/flex2-40.toml
```

At the `$` prompt, type `D` — SWTBUG's disk-boot command:

```
$ D
FLEX 2.0
DATE (MM,DD,YY)? 1,15,80

+++
```

D reads FLEX's cold-loader off track 0 and jumps into it; FLEX signs on, asks the date, and drops to its **+++** prompt. Type **CAT** to see what is on the disk:

```
+++CAT
...directory listing...
+++
```

What actually happened

Nothing was faked, and it is worth knowing what the one command did, because the rest of the manual is built on it.

The machine file named a **SWTPC 6800 with SWTBUG in ROM, an MP-S serial console, and a DC-4 floppy controller**, put the FLEX image in drive 0, and then did one more thing: it typed **RESET** and **RUN** for you. That is what the **startup>** lines are telling you. **There is no **BOOT** command in this program** — on a real SWTPC you pressed RESET, which jumped into SWTBUG, and then typed **D** to boot the disk, so that is what the machine file says. Anything you can type, a machine file can do; it gets no special powers.

From there it is all real: SWTBUG's **D** read the boot sector off track 0, that loader pulled FLEX into memory and jumped into it, and FLEX printed its banner.

Where it found the disk — one base directory

You typed **one** path — the machine file — and the simulator found the rest itself: the disk image, named *inside* that file. Both point at the same place, and knowing where is the whole of how the program handles directories.

- **The machine file on the command line** is found from **your shell** — the directory you were in when you ran the command — because at that instant no machine exists yet. `examples/flex/flex2-40.toml` is walked down from there.
- **Everything after resolves against the machine's own directory** — the folder that `.toml` lives in. The disk it mounts, a PROM it loads, and anything you type at the `swtpcsim>` prompt (`MOUNT`, `LOAD`, `SYMBOLS LOAD`) all come from there. `flex2-40.toml` mounts `FLEX2-40.DSK` with no directory at all, and the simulator looks for it *beside the* `.toml` — and so would a `MOUNT FLEX2-40.DSK` you type.

Here it is concretely. Say you unzipped the package into `~/swtpcsim` and launched it from there:

```
$ pwd
/home/you/swtpcsim
$ ./swtpcsim examples/flex/flex2-40.toml
```

- `examples/flex/flex2-40.toml` — **you** named it to the shell, so it is found from where you stand: `~/swtpcsim/examples/flex/flex2-40.toml`.
- `FLEX2-40.DSK` — the machine mounts it, so it is found in the machine's directory: `~/swtpcsim/examples/flex/FLEX2-40.DSK`. Type `MOUNT dc40:drive0 FLEX2-40.DSK` and you get that same file, from that same folder.

Copy that folder somewhere else and it still boots, because the disk's path is tied to the machine, not to you:

```
$ cp -R examples/flex /tmp/myflex
$ ./swtpcsim /tmp/myflex/flex2-40.toml
```

Your shell never left `~/swtpcsim` — the simulator does not change your working directory — yet the disk is now read from `/tmp/myflex/FLEX2-40.DSK`, beside the machine file you named, and anything you type at the prompt is found there too. That is the whole reason every example is a self-contained folder you can copy anywhere and still boot: the disk, and everything you do to the machine, travels with the machine file.

To see it, ask — **SHOW PATHS** at the `swtpcsim>` prompt prints the base directory: the machine's own folder. What a machine file mounts, and what you *type* (`MOUNT`, `LOAD`, `SAVE`, `DO`, or a `-s` script), both resolve against it. For a built-in machine, which has no folder of its own, it is the directory you launched from.

The `./`, and running from anywhere

Every command so far began with `./swtpcsim` — which means *the swtpcsim in **this** folder*. A fresh unzip drops the program into a directory your shell does not search for commands, so you have to point at it, and `./` is how you say *look right here*.

You can make that unnecessary by **installing** the program — copying it into one of the directories your shell already searches, its **PATH** (on macOS and Linux, `/usr/local/bin` is the usual one). Once it is on

your `PATH` you drop the `./` and type just `swtpcsim`, from any directory at all — including from *inside* an example folder:

```
$ cp swtpcsim /usr/local/bin/      # once: put it on your PATH
$ cd examples/flex
$ pwd
/home/you/swtpcsim/examples/flex
$ swtpcsim flex2-40.toml
```

Now your shell is *inside* `examples/flex`, so the machine file is simply `flex2-40.toml` with no directory — found from where you are — and the disk it mounts is found beside the file, as always. The rule did not change; only where the program lives, and which directory you ran it from, did.

Getting back out — `^E`

Press `^E` (Control-E). This is the **STOP** switch, and it is how you take the keyboard back from a running program:

```
$
STOP -- the machine is still at E216. RUN resumes.
H0I1N0Z1V0C0 A=0D B=00 X=8004 SP=A03D PC=E216 LDAB 00,X
swtpcsim>
```

You are back at the monitor, and **the machine is stopped exactly where it stood**. The processor executes nothing while this prompt is up: the `PC=E216` above is where it will still be in an hour. That is what makes the prompt useful — you can read memory, single-step, and set a breakpoint, and none of it is a moving target.

Stopped is not **lost**. STOP is not RESET and it is not POWER: every register, every byte of memory, the disk in the drive and the CPU's place in its own program are all exactly as they were. That is the whole content of "*the machine is still at E216*" — it is telling you the machine is intact and says where to pick it up.

Why `^E` and not `^C`? Because `^C` belongs to the software running on the machine — so the host intercepts `^E` before the running program is ever offered the byte, and no program inside the machine can take it from you. Everything else, `^C` included, goes straight through. (If `^E` collides with something you need, `CONSOLE stop=1D` moves it to `^J`.)

Going back in — `RUN`

```
swtpcsim> RUN
```

That is all. The machine never stopped, so it simply picks up where it was, and your `$` (or FLEX `+++`) prompt is where you left it.

Leaving — QUIT

```
swtpcsim> QUIT
```

There is no `EXIT`. `Q` will do.

The three things to remember

<code>^E</code>	stop the CPU, back to the monitor. The machine stops where it stands, and loses nothing.
<code>RUN</code>	start the CPU running again.
<code>QUIT</code>	done.

Careful: the disk is real, and there is no undo

The FLEX example mounts its disk **read-only** (`writeprotect = true`) for exactly this reason, so looking around can never dirty the image. When you want to save files back — FLEX's `SAVE`, `NEWDISK`, a file copy — drop that line, and then anything FLEX writes happens to the file on your host with nothing keeping a copy. Two ways to be safe:

- **Write-protect it.** `MOUNT dc40:drive0 examples/flex/FLEX2-40.DSK R0` refuses every write at the controller, so the file cannot change however the program behaves.
- **Copy the folder** when you actually intend to write. It is self-contained and boots from anywhere:

```
$ cp -R examples/flex my-flex
$ ./swtpcsim my-flex/flex2-40.toml
```

No disk? Start with the 680b instead

If you would rather see something simpler than a whole operating system, try the `altair680` machine: a MITS Altair 680b that comes up straight into its ROM monitor, `MON680`, at a `.` prompt — no disk, no boot, just a 6800 and a terminal. The examples chapter walks it through, along with loading a cassette on the 680b's KCACR interface.

Quick reference

Getting out, and back in

Key	Does
<code>^E</code>	STOP — stop the machine and take the keyboard back. Nothing is lost.
<code>RUN</code>	Resume, at the exact instruction it stopped on.
<code>QUIT</code>	Leave. (There is no <code>EXIT</code> .)

Editing the command line

`Tab` completes what you are typing — a command, then a board id, then its property names, then a property's values (`SET mem0 fill=` then `Tab`); a second `Tab` lists the choices. `Up` / `Down` walk the command history, saved per directory in `.swtpcsim_history`.

Key	Does
<code>Ctrl-A</code> / <code>Ctrl-E</code> (or <code>Home</code> / <code>End</code>)	start / end of line
<code>Alt-B</code> / <code>Alt-F</code> (or <code>Ctrl-Left</code> / <code>Ctrl-Right</code>)	back / forward one word
<code>Ctrl-W</code> / <code>Ctrl-K</code> / <code>Ctrl-U</code>	erase word behind / to end of line / whole line
<code>Backspace</code> / <code>Delete</code>	erase before / under the cursor

Command line

```
swtpcsim [options] [machine]
```

machine	a built-in name, or a config file (has a '/' or ends .toml). Omitted: ./swtpcsim.toml if there is one, else `default`.
-m, --machine <n>	ALWAYS a built-in name -- never a file.
-f, --file <path>	ALWAYS a file -- never a built-in name.
-n, --none	empty backplane: no boards, no memory, nothing.
-l, --list	list the built-in machines and exit.
-s, --script <f>	run a command script, then exit with its status. Paths in it are relative to the script's folder.
-x, --exec <cmd>	run one monitor command (repeatable), then exit.
-i, --interactive	after --script/--exec, stay in the monitor.
--mcp	MCP server on stdio.
-v, --version	print the version and exit.
-h, --help	print this help and exit.

Monitor commands

Type the part before the bracket.

Command	Does	Usage
BO[ARDS]	List, add, or remove boards on the backplane.	BOARDS [LIST] ADD <type> <id> [k=v...] REMOVE <id>
B[REAK]	Set a breakpoint on an address, memory/I/O access, or tape stop.	BREAK [<addr> MEM R W <addr> TAPE STOP] [IF <expr> LOADS <expr>] [TRACE ON OFF]
COM[PARE]	Compare a range of memory against another address.	COMPARE <range> <addr>
C[ONFIG]	Load or save the whole machine as a TOML file.	CONFIG LOAD <f.toml> CONFIG SAVE <f.toml>
CONN[ECT]	Attach a serial unit to an endpoint (console, socket, file, ...).	CONNECT <id>:<u> <endpoint>
CONS[OLE]	Show or set the host console's properties.	CONSOLE [<k>=<v>...]
DE[POSIT]	Write bytes into memory at an address.	DEPOSIT <addr> <bytes...>
DI[SASM]	Disassemble memory into instructions.	DISASM [<addr> <range>] [n] [CPU=6800 6809]
DISC[ONNECT]	Unplug the endpoint from a serial unit.	DISCONNECT <id>:<u>
D0	Run a file of monitor commands, one per line, as if typed.	D0 <file>
D[UMP]	Show memory as hex and ASCII.	DUMP [<addr> <range>] [WIDTH=16]
E[DIT]	Enter bytes into memory interactively from an address.	EDIT <addr> [ROM]
EX[AMINE]	Point the front panel at an address (and show that byte).	EXAMINE [<addr>]
F[ILL]	Fill a range of memory with a byte.	FILL <range> <byte>
HE[LP]	Show help for a command.	HELP [<command>]
H[ISTORY]	Replay the recent instruction (or bus-cycle) history.	HISTORY [BUS CPU] [n]
L[OAD]	Load a file into memory (binary, Intel hex or S-record).	LOAD <file> [AT <addr>] [FORMAT=BIN HEX SREC] [ROM]
MA[CHINE]	Load a built-in machine by name (MACHINE none empties the backplane).	MACHINE <name> MACHINE none

M[OUNT]	Put a disk or tape image into a drive; a .imd is converted to a raw .dsk beside it.	MOUNT <id>[:<u>] <file> [WP] [CREATE] [extract[=<base>]] [k=v...]
MOV[E]	Copy a range of memory to another address.	MOVE <range> <dest> [ROM]
N[EXT]	Step one instruction, running any JSR/BSR to completion.	NEXT
NO[BREAK]	Remove a breakpoint, or all of them.	NOBREAK [id]
P[OWER]	Power-cycle the machine -- the only thing that clears RAM.	POWER
Q[UIT]	Leave the simulator.	QUIT
REGI[ON]	Add a RAM or ROM region to a memory board.	REGION ADD <id> type=ram rom at=<addr> [size= mount=]
RE[GS]	Show the CPU registers (SET REG changes one).	REGS SET REG <r>=<v>
RES[ET]	Reset the machine, keeping RAM (RESET CPU resets just the processor).	RESET [CPU]

REST[ORE]	Load machine state back from a snapshot.	RESTORE <file>
R[UN]	Start or resume the machine, optionally at an address.	RUN [addr]
SA[VE]	Write a range of memory out to a file.	SAVE <file> <range> [FORMAT=BIN HEX OCTAL PRN]
SEA[RCH]	Find bytes or a string in a range of memory.	SEARCH <range> <bytes...> "str"
SE[T]	Change a property of a board, the console, display, a register, the bus, or the machine.	SET <id>[:<u>] CONSOLE DISPLAY MACHINE REG BUS <k>=<v>
SH[OW]	Display the state of a board, the bus, or the machine.	SHOW <id> BOARDS BOARD <type> [UNITS] MACHINES MACHINE [<name>] BUS [MAP IRQ CONTENTION] ROMS MOUNTS PATHS CONSOLE DISPLAY SYMBOLS CLOCK VERSION
SN[APSHOT]	Save the whole machine state to a file.	SNAPSHOT <file>
STA[RTUP]	Edit the machine's boot list (the commands CONFIG SAVE writes as startup = [...]).	STARTUP [ADD <command> REMOVE <n> CLEAR]

S[TEP]	Run one instruction (or n), showing the registers after each.	STEP [n]
SY[MBOLS]	Load or clear a symbol table for disassembly.	SYMBOLS LOAD <file> [REPLACE] SYMBOLS CLEAR
T[RACE]	Log every bus cycle while the machine runs.	TRACE ON OFF [file] [MASK=IRQ,CONTENTION]
TY[PE]	Feed text to the guest as if typed at its keyboard.	TYPE "text"
U[NMOUNT]	Take a disk or tape out of a drive.	UNMOUNT <id>:<u>
W[H0]	Say which board answers an address or I/O port.	WHO <addr>

Boards

CPU

Type	What it is
6800	Altair 680b / SWTPC CPU board: a Motorola 6800
6809	CPU board: a Motorola 6809

Memory

Type	What it is
memory	RAM/ROM board: plain, unbanked memory regions

Disk

Type	What it is
dc4	SWTPC DC-4: WD179x 5.25" floppy controller

Serial

Type	What it is
680io	Altair 680b onboard I/O: 6850 console and strap port
mps	SWTPC MP-S: one 6850 serial port on an SS-30 slot

Tape

Type	What it is
680kcacr	Altair 680b KCACR: Kansas City audio cassette

Parallel and printer

Type	What it is
680uio	Altair 680b Universal I/O: 6850 serial port and 6820 PIA

Machines

Machine	What it is
altair680	The Altair 680b -- MITS's second machine, and a different animal from the 8800.
swtpc	The SWTPC 6800 -- Southwest Technical Products' 1975 computer, the machine this simulator is named for.

A machine file, in one look

```
[machine]
name      = "mine"
base      = "default"      # start from a machine, and say what is DIFFERENT
startup   = ["RUN FF00"]    # the operator's own keystrokes. There is no BOOT verb.

[[board]]                                # type + a NEW id      -> ADD the card
type      = "mps"           # type + an id from the base -> REPLACE it outright
id        = "mps0"          # NO type + an id      -> MODIFY the one already there
at        = 8004             # remove = true       -> PULL THE CARD OUT

[board.unit.tty]                         # a unit's own settings
connect   = "console"

[[board.region]]                         # a list the card owns (memory)
type      = "ram"
at        = 0000             # hex: it is an address
size      = "32K"           # decimal: it is a size

[[board.drive]]                          # a list the card owns (disk controllers)
unit      = 0
mount     = "flex.dsk"       # relative to THIS FILE

[console]                               # the HOST's terminal -- not a board
stripout  = true
base      = octal           # read/print the wire class in split octal
```

Paths: a path *inside* a machine file is relative to **that file**. A path you type is relative to **your shell**.

Endpoints — `CONNECT <id>:<unit> <endpoint>`

Endpoint	Is
<code>console</code>	the host terminal. Exactly one unit may hold it.
<code>null</code>	nowhere. Writes vanish, reads never come.
<code>loopback</code>	itself — what you write comes back.
<code>scripted</code>	a caller in place of a human — what MCP and the tests type into.
<code>socket:PORT</code>	listens , as a raw pipe — for a program at the far end. <code>?banner</code> greets each caller.
<code>socket:HOST:PORT</code>	calls out , as a raw pipe.
<code>telnet:PORT</code>	listens , speaking Telnet — this is telnet-in for a person. Greets each caller; <code>?banner=off</code> stops it.
<code>telnet:HOST:PORT</code>	calls out , taking the telnet client's part.
<code>serial:DEVICE</code>	a real serial port on this host.
<code>in:PATH</code>	a host file as a reader (paper tape). <code>?cps=N</code> paces it.
<code>out:PATH</code>	a host file as a punch — 8-bit clean, never truncating.
<code>terminal</code>	a window the simulator draws itself (SDL builds). <code>?emulation=vt100 adm3a vt52 h19</code> , <code>?size=COLSxROWS</code> .
<code>printer:QUEUE</code>	a real print queue on this host.
<code><endpoint> FILE</code>	a tap: append <code> FILE</code> to any endpoint to also log the line.
<code><endpoint> socket:PORT</code>	a live mirror: <code>telnet</code> in to watch and take over. <code>?ro</code> = watch-only.

Machines

A **machine** is a backplane with boards in it. Which boards, with what settings, in what state — that is all a machine is, and it is the only thing `swtpcsim` needs to be told.

You tell it in one of three ways: name a **built-in**, name a **file**, or say **nothing** and take the default. This chapter is about how that choice is made, and about the one rule that decides what a path means.

The command line

```
swtpcsim [options] [machine]
```

Option	
<code>machine</code>	a built-in name, or a config file if it contains a <code>/</code> or ends in <code>.toml</code>
<code>-m, --machine <name></code>	always a built-in name — never a file
<code>-f, --file <path></code>	always a file — never a built-in name
<code>-n, --none</code>	an empty backplane. No boards, no memory, nothing
<code>-l, --list</code>	list the built-in machines and exit
<code>-s, --script <file></code>	run a command script, then exit with its status. Paths in it are relative to the script's folder
<code>-x, --exec <cmd></code>	run one monitor command, then exit. You can give it more than one time
<code>-i, --interactive</code>	after <code>--script</code> or <code>--exec</code> , stay in the monitor
<code>--mcp</code>	run as an MCP server on stdio
<code>-v, --version</code>	print the version and exit
<code>-h, --help</code>	print this help and exit

Give exactly one machine. A positional name *and* a `-m`, or a `-f` *and* a `-n`, is an error — the program says *give ONE machine* and stops. It does not guess which one you meant.

How a bare word resolves — and why it never looks at your disk

```
$ swtpcsim swtpc           a BUILT-IN, by name
$ swtpcsim examples/flex/flex2-40.toml  a FILE, by path
```

The rule is **purely syntactic**. The filesystem is **never probed**:

The word	What it is
contains <code>/</code> or <code>\</code>	a file
ends in <code>.toml</code>	a file
anything else	a built-in name

That is deliberate, and it is worth being clear about why, because a simulator that guessed would be more convenient exactly until the day it was not.

`swtpcsim swtpc` means **swtpc in every directory on earth**. It means the same thing on your machine and on mine. It cannot be hijacked by a file called `swtpc` that happens to be lying next to you — because the program never asks whether such a file exists. A command in a script, a line in a README, a habit in your fingers: all of them keep meaning what they meant.

The cost is that `swtpcsim mymachine.toml` needs the extension, and `swtpcsim ./mymachine` needs the `./`. That is a small price, and if you want the question settled explicitly, settle it:

```
$ swtpcsim -m swtpc          a built-in. Stop looking for a file
$ swtpcsim -f ./swtpc       a file called `swtpc`, no extension, right here
```

`-m` and `-f` are how you say what you mean when the syntax will not say it for you.

The one file the simulator *finds*

If you name **nothing at all**, and the working directory contains a file called `swtpcsim.toml`, that machine is loaded — and it says so:

```
$ swtpcsim
swtpcsim: no machine named -- using ./swtpcsim.toml (`-m default` for the built-in).
swtpcsim 1.0.0 -- 6800, full speed.
machine: bench.  HELP for commands.
swtpcsim>
```

The **first line is the announcement**, and it is printed before anything else so you cannot miss it. The `machine:` line after it names the machine *the file* declares — `bench` here, not the file it came out of — so the two lines together say both halves: where it came from, and what it turned out to be.

This is the **only** file the simulator finds rather than is given, and it only happens when the command line names nothing whatsoever. Name a built-in, a file, or `-n`, and `./swtpcsim.toml` is ignored — you asked for something, so you get it.

Put one in a project directory and `swtpcsim`, bare, is your machine. It **announces itself when it does**, so you are never running something you did not know about.

With no `swtpcsim.toml` and no arguments, you get the built-in `default`.

The built-in machines

A **built-in** is a TOML machine file compiled into the binary. There is nothing privileged about it: same format, same keys, same rules as one you write yourself. It is in the program only so that it is always there.

```
$ swtpcsim --list
```

names them, one to a line, each with a sentence saying what it is — and it is the live list, so it cannot be short of a machine the way a list typed into a chapter can. The machine reference at the back of this manual is that same table. From the `swtpcsim>` prompt the list is `SHOW MACHINES`.

To see what is in one — its backplane and its startup — name it:

```
$ swtpcsim -x 'SHOW MACHINE' swtpc
```

or, at the prompt, `SHOW MACHINE swtpc`. A bare `SHOW MACHINE` there is the machine you are running now.

And to get it as **text you can edit** — the actual machine file, every board, every setting:

```
$ swtpcsim -x 'CONFIG SAVE mine.toml' swtpc
$ swtpcsim mine.toml
```

`CONFIG SAVE` writes the machine you are actually running, and it round-trips. **Which makes every built-in a worked example.** Find the one closest to what you want, save it out, and edit it.

Or better, do not copy it at all — start *from* it with `base`, and write down only what is different. The configuring chapter is about that.

The empty backplane — `-n`

```
$ swtpcsim -n
```

No boards. No memory. No processor. `-n` is a bare chassis, and every `BOARDS ADD` from there is yours. It is the honest starting point when you are building a machine up board by board, and it is the one way to be certain nothing is in there that you did not put there.

The path rule: one base directory

This is the rule that lets an example directory be copied anywhere and still boot. It is one sentence:

A relative path resolves against the machine's directory — the folder the machine file was loaded from.

That folder is the base for the disks and PROMs that the machine file mounts. It is also the base for the `MOUNT`, `LOAD`, `SAVE` and `D0` paths that you type. The rule is the same for a path in the file and a path that you type.

A **script** follows the same idea. A path in a script is relative to the script's folder, as a path in a machine file is relative to the machine file. This is true for a script that you run with `D0` and for a script that you run with `-s`.

When `examples/flex/flex2-40.toml` says `mount = "FLEX2-40.DSK"`, it means *the disk in this folder* — and it goes on meaning that after you copy the folder to your desktop, rename it, or mail it to someone. That is why the examples are self-contained directories, and why the quick start's `cp -R` actually works. And when you then type

```
swtpcsim> MOUNT dc40:drive1 FLEX2-40.DSK
```

you get the **same file**, from the **same folder** — the one the machine came from — no matter which directory you launched `swtpcsim` from. Typed paths used to resolve against your shell instead, which is how the identical disk could show up under two different names; that split is gone.

A **built-in** machine has no directory of its own, so its base is the directory you launched from — the only anchor it has.

When it bites, and what it looks like

The rule is invisible until a file is missing, and then it can look like a typo that is not one. Keep your machine files in a `machines/` folder, write a path meaning *the folder you launched from*, and you get the one confusing case:

```
[[board.drive]]
unit = 0
mount = "disks/flex/FLEX2-40.DSK" # meant: the disks/ up beside machines/
```

`swtpcsim -f ./machines/swtpc.toml` then says:

```
./machines/swtpc.toml: dc40: 'machines/disks/flex/FLEX2-40.DSK': no such file
('disks/flex/FLEX2-40.DSK' is relative to the machine's directory, ./machines/)
```

The disk is not missing. It was looked for beside the machine file, because that is the machine's directory and that is where relative paths point. Write it the way the machine sees it:

```
mount = "../disks/flex/FLEX2-40.DSK" # up out of machines/, then down into disks/
```

...or keep the machine file next to what it mounts, which is what every shipped example does. The same `../` applies whether the path is in the file or you type it — because both resolve against the one base.

Ask the machine, rather than working it out

You do not have to hold this in your head. `SHOW PATHS` prints the base, for the machine you are actually running:

```
swtpcsim> SHOW PATHS
base directory    /home/you/swtpc/flex
                  What a machine file mounts, and the MOUNT / LOAD / SAVE /
                  DO you type, resolve against this. A path inside a DO or
                  -s file is relative to that file.
                  It is the directory the machine was loaded from.
```

The base is where paths point — what a machine file mounts and what you type both resolve against it.

Boot a **built-in** machine and the base is the directory you launched from, because a built-in carries no folder of its own:

```
base directory    /home/you/swtpc
...
                  This machine is built in, so it is the directory you
                  launched from.
```

`SHOW MOUNTS` is the companion: every disk, tape and ROM in the machine and what is in each, across all the boards at once.

```
swtpcsim> SHOW MOUNTS
UNIT      KIND  HOLDS
dc40:drive0 disk  FLEX2-40.DSK
dc40:drive1 disk  (empty)
dc40:drive2 disk  (empty)
dc40:drive3 disk  (empty)
mem0:rom0  rom   builtin:swtbug (read-only)

Paths are AS WRITTEN.  SHOW PATHS says what they are relative to.
```

Empty drives are listed, not hidden. The DC-4 has four, one disk is in it, and the other three doors are open — which is the machine, and worth seeing.

That last line is the command telling you what the middle column is worth, and it is why the two belong together: `SHOW MOUNTS` tells you what the machine was told, and `SHOW PATHS` tells you what that meant.

None of this is a sandbox

The path rule decides **where a path points**, and confines nothing. A machine file may mount any file on your disk — with `..`, or with an absolute path — and it will be opened. It is a rule about resolution, not about permission.

Running a command and leaving — `-x` and `-s`

`swtpcsim` does not have to be interactive.

```
$ swtpcsim -x 'SHOW MACHINE' default
$ swtpcsim -x 'DUMP 0 F' examples/flex/flex2-40.toml
```

`-x` runs one monitor command against the machine and exits. It is **repeatable**, and the commands run in the order you gave them:

```
$ swtpcsim -x 'MOUNT dc40:drive0 mine.dsk' -x 'RUN' -i examples/flex/flex2-40.toml
```

`-i` is the difference between a query and a start-up: without it the program exits when the commands are done; with it you are dropped into the monitor with the machine exactly as your commands left it. `-i` alone, with no `-x` or `-s`, does nothing.

`-s` runs a **script** — a file of monitor commands, one per line, the same ones you type:

```
$ swtpcsim -s boot.cmd examples/flex/flex2-40.toml
```

A path in a script is relative to the script's folder, as a path in a machine file is relative to the machine file. So a script that mounts a disk beside it works from any folder: give the path to the script from the folder that you are in.

The exit status is non-zero if any command failed. That is the whole point: `swtpcsim -s` is a program you can put in a shell script, a Makefile, or a build, and test the result of.

```
if swtpcsim -s check.cmd mine.toml; then
    echo "machine is sane"
fi
```

Which chapter next

The **configuring** chapter is the machine file itself: every table, every key, and the four things a `[[board]]` entry can mean. The **boards** chapter is what the boards *are*.

The machine file

A machine file is **TOML**. It lists the boards in the backplane, what is set on each of them, and what to do once the power is on. That is all it does, and there is nothing else it can do.

This chapter is the definitive description of the format.

What TOML is

The machine file is written in **TOML** — a plain configuration format meant to be written by hand and read back a year later without a manual. You need only a handful of rules to read every example below:

- **key = value** — one setting per line: `clock_hz = 20000000`.
- **Quotes are for text**. A string is quoted (`name = "flex"`); a number or a true/false is bare (`size = 256`, `idle = true`).
- **[table]** appears once — `[machine]` is the machine, `[console]` is your terminal.
- **[[table]]** — doubled brackets — repeats. Each `[[board]]` starts another board.
- A nested `[board.unit.x]` belongs to the block above it, by name — the indentation in these examples is only for the eye.
- **#** starts a comment to the end of the line. A comment that begins `#>` is a *note* — the file prints it to you when it loads (see below).

That is the whole of the syntax. Everything below is which keys go where.

The one thing to know first

Anything you can type, a config can do — and nothing more. A machine file has no special powers. It cannot boot a disk, because there is no `BOOT` verb; what it can do is *type* `RESET` and `RUN` for you, which is what the operator did to bring the ROM monitor up. Every board setting in a machine file is a setting you could have made with `SET` at the prompt, and every `SET` you make at the prompt is a key you could have written in the file. **A board's properties are its TOML keys.**

There is no separate config schema anywhere in the program. That is why the board reference at the back of this manual is exhaustive, and why it cannot drift out of date: it is printed from the same table the monitor resolves against.

Nothing is silently ignored

Any unknown table or key is a hard error, with a sentence saying so. The machine does not load.

```
mine.toml: unknown [machine] key 'widget'
mine.toml: [[board]] cpu0: cpu0 has no property 'frobnicate'. Known: clock_hz idle achieved_hz
```

This is not fussiness. A configuration that looks like it set something and did not is worse than one that will not load, because you will spend the afternoon debugging the machine instead of the typo. A misconfiguration in this program cannot be silent.

Notes the operator sees — #>

An ordinary `#` comment is for whoever *reads the file*. A comment that begins `#>` is for whoever *loads it*: its text is printed to you when the machine comes up, right after the `loaded ...` line and before the `startup` commands run. It is where the file's author leaves the one or two sentences you need to actually use the machine.

```
#> Boots FLEX 2.0 from drive 0.
#> At SWTBUG's $ prompt type D to boot the disk, then CAT at the +++ prompt.

[machine]
name = "flex"
base = "swtpc"
startup = ["RESET", "RUN"]
```

- One `#>` line is one printed line. Write as many as you like, in a row or scattered through the file — they print in the order they appear.
- `#>` on its own prints a blank line, so you can space a note into a short paragraph.
- A `#>` can trail a setting, too: `name = "flex" #> the 40-track variant`.
- Under `--mcp` the notes go to `stderr`. There `stdout` carries only the MCP messages, so a client reading it sees nothing else; you still see the notes in the terminal.
- It is still a comment. It **sets nothing**, and `CONFIG SAVE` does not write it back — a saved machine is the backplane, not the prose around it. If a note is worth keeping, keep it in the file you wrote by hand.

An ordinary `#` comment, as always, is seen by no one but the reader of the file.

The tables

Table	What it is
[machine]	the machine's identity. Three keys, no more
[[board]]	a board. One entry per board
[board.unit.<name>]	one unit <i>on</i> the board above — a serial channel, a tape deck
[[board.region]]	a memory region on a <code>memory</code> board
[[board.drive]]	a drive on a disk controller
[console]	your terminal . Not a board — see below

[machine] — and it has exactly three keys

```
[machine]
name      = "flex"
base      = "swtpc"
startup   = ["RESET", "RUN"]
```

name

What the machine is called — what `SHOW MACHINE` prints and the video window's title bar shows. That is the whole of it. `SET MACHINE name=` changes it at the prompt.

base — start from a machine, and say what is *different*

```
base = "swtpc"           # a built-in
base = "../flex/flex2-40.toml" # or a file
```

The value resolves by **the same syntactic rule as the command line**: contains a `/` or ends in `.toml` → a file; otherwise → a built-in name. (The machines chapter explains why the filesystem is never probed.) A file path is relative to *this* file.

Two rules about where it goes:

- **base** is processed before every other key, whatever order the file is written in.
- **base** must appear before the first `[[board]]`. You cannot inherit a backplane you have already started modifying.

`base` nests **up to 8 deep**. A machine built on a machine built on `swtpe` is fine.

This is the key that makes the format worth using. Without it, every variant of a machine is a copy of a hundred lines, and the day you change one of them you change it in six files. With it, a machine file says only **what is different**, and reads as the diff it actually is.

startup — the operator's keystrokes, written down

```
startup = ["RESET", "RUN"]
```

An array of **ordinary monitor commands**, run once the machine is built. Any command. They run in order, and you see them run — that is the `startup>` line in the quick start.

Paths inside a `startup` command are relative to the machine file — the machine's own directory, which is the base every relative path uses, whether written in the file or typed at the prompt. So a `startup` line and the same command typed by hand find the same file.

What `[machine]` will *not* take

```
[machine]
clock_hz = 1000000      # ERROR
```

It is rejected, with an explanation:

```
mine.toml: clock_hz belongs to the CPU BOARD, not to [machine] --
the crystal is on the board. Put it in the CPU's [[board]]:
  [[board]]
  type    = "6800"
  id      = "cpu0"
  clock_hz = 1000000
```

The crystal is soldered to the **CPU board**. It is not a property of "the machine" — the machine is just the box the board is plugged into. If you pull the CPU board out, the crystal goes with it.

It gets a custom error rather than the generic *unknown key* because it is the one people reach for first, and being told *where the thing actually lives* is more use than being told it isn't here.

[[board]] — and it has four forms

This is the heart of the format. What a `[[board]]` entry means depends on whether it has a `type`, and whether its `id` is one the base already used.

Write	And it means
<code>type</code> + a new <code>id</code>	ADD the board
<code>type</code> + an <code>id</code> from the base	REPLACE the board outright
<code>no type</code> + an <code>id</code>	MODIFY IN PLACE
<code>remove = true</code> + an <code>id</code>	PULL THE BOARD OUT

`id` is always mandatory. It is how you refer to the board at the prompt, and how a later file refers to it here.

ADD — `type` + a new `id`

```
[[board]]
type = "680uio"
id   = "uio0"
```

A board that was not there is now there. In a file with no `base`, this is the only form — there is nothing to modify, replace or remove.

REPLACE — `type` + an `id` the base already used

```
[[board]]
type = "mps"
id   = "mps0"
base = 0x8008
```

If the base had a board called `mps0`, it is **gone** — pulled out and thrown away — and a fresh `mps` is fitted in its place. **Everything the base set on that board is lost**, including the settings you did not mention. You get the type's defaults, plus whatever you write here.

That is what "replace" means, and it is almost never what you want. You want:

MODIFY IN PLACE — no `type`

```
[[board]]
id    = "cpu0"
clock_hz = 1000000
```

Leave the `type` out and you are reaching into the board that is already there. Everything the base set on `cpu0` stays set; you change the crystal and nothing else.

The absence of `type` is the whole signal. It reads oddly for about a day and then reads as exactly what it is: *I am not fitting a board, I am adjusting one.*

REMOVE — `remove = true`

```
[[board]]
id      = "uio0"
remove = true
```

The board is pulled out of the backplane. Its ports stop being decoded. Nothing else in the file may mention it.

The error that catches a copy-paste

`type` + an id that *this same file* has already declared is an error. Not a replace — an error. Within one file, declaring the same board twice is never something you meant; it is a block you copied and forgot to rename. The file will not load, and it will tell you which id.

(Across files it is different: an id from your *base* is a board you inherited, and replacing it is a legitimate thing to want.)

Everything else on a `[[board]]` is a property

`type`, `id` and `remove` are the only keys the config layer understands. Every other key is handed straight to the board.

```
[[board]]
type = "mps"
id   = "mps0"
base = 0x8004      # the mps knows what a slot base is. The config layer does not.
```

The config layer knows nothing about addresses, baud rates or drive counts. It cannot, and it does not try. It routes the key to the board and the board accepts it or rejects it by name:

```
mine.toml: [[board]] cpu0: cpu0 has no property 'frobnicate'. Known: clock_hz idle achieved_hz
```

The full key list for every board is the board reference at the back of this manual. The boards chapter says what the boards *are*.

[board.unit.<name>] — settings that belong to one unit

Some boards carry more than one independent thing. A 680b Universal I/O board carries a **6850 serial port** and a **6820 PIA** — different chips, not two channels of one: the serial unit and the PIA section have their own settings, their own endpoint, and they share nothing at all. So they get their own tables.

```
[[board]]
type = "680uio"
id   = "uio0"
base = 0xF000          # the BOARD's property -- the window both chips live in

[board.unit.serial]
baud   = 9600          # the serial port's property
connect = "console"

[board.unit.pia]
connect = "out:pia.log" # the PIA's section A is a different chip. It does not care.
```

A key in `[board.unit.serial]` is exactly the key `SET uio0:serial baud=9600` takes at the prompt. It is the same property, reached two ways.

The board reference lists which boards have units, and what each unit takes.

[[board.region]] — memory

A `memory` board is a **list of regions**, which is why one physical card can carry 32K of RAM and the monitor PROM up in high memory. The regions are the board.

```

[[board]]
type = "memory"
id   = "mem0"

[[board.region]]
type = "ram"
at   = 0x0000      # HEX -- it is an address
size = "32K"       # DECIMAL -- it is a count

[[board.region]]
type = "rom"
at   = 0xE000
mount = "builtin:swtbug" # a file path, or builtin:<name>

```

Key	
type	required. ram or rom
at	the address where the region starts. Hex
size	the size of a ram region. Decimal. You can use the K and M suffixes. A rom region takes its size from its image, rounded up to a page, so it ignores size
mount	a ROM image: a file path, or builtin:<name>

(A size with a suffix is written as a string — `size = "56K"` — because `56K` is not a number TOML will accept bare. A plain count needs no quotes: `size = 256`.)

An empty socket

A **rom** region with no **mount** is an **empty socket**, even when it has a `size`. It decodes nothing, and reads there float to `FF` — because that is what a bus with nobody driving it does. It is not zeros, and it is not an error. It is an unpopulated socket on a board that has one, which is a thing a real machine could be, and software that reads it gets `FF`.

[[board.drive]] — disks

A disk controller addresses drives; a drive holds an image.

```
[[board]]
id = "dc40"

[[board.drive]]
unit      = 0           # DECIMAL -- it is a drive number
mount     = "FLEX2-40.DSK" # relative to THIS FILE
readonly  = false
```

Key	
<code>unit</code>	the drive number. Decimal
<code>mount</code>	the image file
<code>readonly</code>	refuse every write at the controller, so the host file cannot change. For a disk you mean to read — see the disks chapter. <code>writeprotect</code> is the same key under the name the rest of the program uses; write either
<code>media</code>	force a format instead of probing the image
<code>create</code>	make the file, empty, if it is not there — then mount it. <code>MOUNT ... CREATE</code> , written down

`media` is the escape hatch. The controller normally works out the format from the image, and normally it is right; when it is not — a headerless image, an unusual geometry — you say so. It is also what says how big a **blank** disk is, since a blank one matches no format at all. The disks chapter covers the formats, and `create` .

Without `create` , a `mount` naming a file that is not there is an **error and the machine does not load** — the same rule as everywhere else here, that a thing which looks like it worked and did not is the worst outcome available.

Numbers: hex on the wire, decimal for counts

The machine file follows the same number rule as the monitor (the *Monitor* document states it in full): **anything the 6800 sees on the bus is hex; anything that never reaches the bus is decimal.**

```
base = 8004      # 0x8004 -- an address is on the wire, so it is HEX
at    = 0xE000   # an address
baud  = 9600     # a rate -- DECIMAL
size  = 56       # a count -- DECIMAL
```

base = 8004 is address **0x8004** , not eight thousand and four. That is the line that catches people: a base address is on the wire, the wire is hex, and every SWTPC listing wrote the console slot as 8004. In your own files, be explicit — the forcing markers all work here: `0x8004` / `$8004` / `8004h` for hex, `0o...` / `...q` for octal, `0b...` for binary, `#...` for decimal, and a `K` / `M` suffix is always a decimal count. To read and print in octal throughout, set `[console] base = octal` (below) — that changes the base, not

the rule. Board settings keep their own base regardless, so `SHOW` prints an address as `0x8004` however you wrote it.

[console] — your terminal, which is not a board

```
[console]
stop      = 0x05      # STOP: the key that gets you back to the monitor. ^E (attn= still accepted)
base      = hex       # hex | octal -- how you read/write addresses, ports, bytes
upper     = false
strip7in  = false
strip7out = false
crlf      = false
echo      = false
bell      = true
bsdel     = "off"
```

`[console]` is **not** a `[[board]]` and it is not in the backplane. It describes *the terminal you are sitting at* — a piece of equipment on your desk, on the far end of a cable, in 2026. The machine never knew anything about it.

Key	
stop	the STOP key's byte. Hex . Default <code>05</code> = <code>^E</code> . (<code>attn</code> is an accepted alias.)
base	<code>hex</code> <code>octal</code> — how the monitor reads and prints the wire class (addresses, bytes). <code>octal</code> is split octal
upper	fold input to upper case
strip7in	clear bit 7 of everything the guest receives
strip7out	clear bit 7 of everything the guest sends
crlf	translate line endings
echo	echo typed characters locally
bell	let the guest ring your terminal's bell
bsdel	<code>off</code> <code>bs</code> <code>del</code> — what your Backspace key sends

Because they are the terminal's, they reach as far as the terminal does and no further: send a board's console unit somewhere else — `CONNECT mps0:tty socket:2323`, or out a real serial port — and the far end gets the bytes exactly as the guest wrote them, all eight bits. The settings are not undone; the byte just no longer passes through the console, and every line in the machine is 8-bit clean. The serial chapter has the full story, and what to set instead.

The transform chain belongs to the console

The [console] keys above (`strip7out`, `upper`, `crlf` ...) are the **only** thing in the simulator that alters a byte, and they belong to the console because that is where a human is reading text. **Every serial line is 8-bit clean** — there is no bit-masking strap on any board, because a line may carry XMODEM and a line that eats bit 7 is a line you cannot trust with a file. So the fix for a guest whose output arrives garbled with high-bit characters is `strip7out` on the console, never a 7-bit strap on the board. The serial chapter explains why in full.

A complete machine file

Small, whole, and it works. No `base` — so every board is an ADD:

```

[machine]
name      = "tiny"
startup   = ["RESET", "RUN"]

[[board]]
type      = "6800"           # the CPU is a board. The crystal is on it.
id        = "cpu0"
clock_hz  = 0                # 0 = flat out. This is the default.

[[board]]
type      = "mps"           # the console board
id        = "mps0"
base      = 8004             # HEX. Address 0x8004, the SS-30 console slot.

[board.unit.tty]
baud      = 9600             # DECIMAL. Nine thousand six hundred.
connect   = "console"

[[board]]
type      = "memory"
id        = "mem0"

[[board.region]]
type      = "ram"
at        = 0x0000
size      = "32K"

[[board.region]]
type      = "rom"
at        = 0xE000
size      = 1024
mount     = "builtin:swtbug"

```

A **base** delta — and this is the one you will actually write

This is genuine. It is how the FLEX example is built, and it is nine lines:

```

[machine]
name      = "flex"
base      = "swtpc"          # a 6800 CPU, an MP-S console, a DC-4 floppy
                                # controller, RAM and SWTBUG in ROM
startup   = ["RESET", "RUN"] # the operator's own keystrokes, written down

[[board]]
id = "dc40"                  # NO type: modify the controller the base already has

[[board.drive]]
unit      = 0
mount     = "FLEX2-40.DSK"   # relative to THIS FILE

```

Everything a SWTPC 6800 is, `swtpc` already was. The only thing this file has to say is *which floppy is in drive 0*, and *press RESET and RUN* to reach SWTBUG (from there `D` boots the disk). **That is the whole of the difference, and so that is the whole of the file.**

Note what is not here: no `type` on the `[[board]]`, because `dc40` already exists and we are adjusting it, not fitting it. Had we written `type = "dc4"`, we would have thrown the base's controller away and got a fresh one with default settings — and it would still have worked, and we would never have known we had done it. Leave the `type` out.

Saving and loading at the prompt

```

swtpcsim> SET MACHINE name=mine
swtpcsim> CONFIG SAVE mine.toml
swtpcsim> CONFIG LOAD mine.toml

```

CONFIG SAVE writes the machine you are actually running — every board, every property, as it stands right now, including everything you changed with `SET` since you started. It **round-trips**: load what it wrote and you get the machine back.

Name it before you save it. A machine keeps the name it was built from — `none` if you started with `-n`, `swtpc` after `MACHINE swtpc` — and that is what lands in the file's `name` line. `SET MACHINE name=` changes it. The name is also what `SHOW MACHINE` prints and what the video window's title bar shows.

CONFIG LOAD is the whole machine, so it replaces the one you have — the same thing that naming the file on the command line does, and there is no undo but the file you saved it to. It is also **all or nothing**: the machine is built off to one side first, so a file that will not load leaves you exactly where you were rather than halfway between two machines.

Which makes it the fastest way to write a machine file. Build the machine at the prompt with `BOARDS ADD` and `SET` until it is what you want, then save it, then edit the file down to the parts you care about — or

give it a `base` and delete the rest.

The one part of the file that is not a board is the **startup list** — the commands the machine runs on load, `MOUNT` the disk, `RESET`, `RUN`. You can build that at the prompt too, with `STARTUP`:

```
swtpcsim> STARTUP ADD MOUNT dc40:drive0 "FLEX2-40.DSK"
swtpcsim> STARTUP ADD RESET
swtpcsim> STARTUP ADD RUN
swtpcsim> STARTUP
  1 MOUNT dc40:drive0 "FLEX2-40.DSK"
  2 RESET
  3 RUN
```

`STARTUP ADD` appends a line exactly as you typed it — quotes and spaces and all, because a startup entry is just a command line; `STARTUP REMOVE <n>` drops one and `STARTUP CLEAR` empties the list. Whatever you assemble is what `CONFIG SAVE` writes as `startup = [ ... ]`, so the boot sequence you tried at the prompt is the boot sequence the file carries.

Boards

A machine is a **backplane** with boards in it — the memory, the serial ports, the disk controller, and the processor itself. There is no "machine" underneath the boards doing the real work; take the boards out and there is nothing left but a bus.

`swtpcsim` is built that way on purpose, and it is the reason the CPU's crystal is a property of the CPU *board* rather than of the machine. Not nitpicking: it is what lets you pull a board out, put a different one in, and find out what the software does about it.

This chapter says what the boards **are** — what the real hardware was, what it is for, and what will bite you. **It does not list their parameters.** Every key of every board is in the board reference at the back of this manual, printed from the program's own tables, which is why it cannot be wrong.

The boards

Grouped by what they do — the same order as the sections below.

Memory

Type	What it is
<code>memory</code>	RAM and ROM, as a list of regions

Processor

Type	What it is
<code>6800</code>	a Motorola 6800 — the CPU of the SWTPC 6800 and the MITS Altair 680b
<code>6809</code>	a Motorola 6809 — a processor board for a machine that you build

Serial ports and consoles

Type	What it is
<code>mps</code>	SWTPC MP-S — a 6850 serial console on the SS-30 bus. The board SWTBUG talks to
<code>680io</code>	Altair 680b onboard I/O — a 6850 serial console and the configuration straps
<code>680uio</code>	Altair 680b Universal I/O — a second 6850 serial port and a 6820 parallel port

Cassette

Type	What it is
680kcacr	Altair 680b KCACR — the Kansas City audio-cassette interface

Floppy

Type	What it is
dc4	SWTPC DC-4 — a WD179x floppy controller. FLEX boots from it

Memory

memory — RAM and ROM

A memory board is a **list of regions**, and the regions are the board. That is not a modelling convenience; it is what a memory board was. One physical card carried banks of chips decoding whatever ranges its jumpers said, and a card with RAM low and a ROM monitor high is a perfectly ordinary card.

So the `swtpc` machine has a `memory` board that carries its RAM, its scratchpad RAM, and SWTBUG in ROM all at once; `altair680` has one carrying a kilobyte of RAM and MON680 in a PROM.

The reset vectors live at the top of memory

A 6800 fetches its RESET, IRQ, SWI and NMI vectors from the very top of the address space — `FFF8 – FFFF`. A ROM monitor has to answer there, and SWTBUG does it by being **mirrored**: the 1 KB image sits at `E000`, and the same image is placed a second time just below the top of memory so a vector fetch at `FFF8 – FFFF` reads the vectors the chip stored at `E3F8 – E3FF`. A region can ask for that second placement with `relocate` — see the configuring chapter — and it is how `swtpc` comes up running SWTBUG's RESET entry.

Two boards, one address — contention

Two boards can decode the same address, and when they do, **both drive the bus and the monitor reports contention**, naming both and the page — the same fault a real backplane would have handed you. The bus arbitrates nothing and picks no winner; a machine that boots is one whose boards each own their own addresses, which is what the built-in machines do.

The CPU board — it plugs in, and it drives the bus

6800 — a Motorola 6800

The processor is a board like any other. It plugs into the backplane, it can be removed, and with `-n` you can build a machine that does not have one. What it does is **drive the bus** — which makes it unlike every other board in the box, and is exactly what a CPU card did.

The 6800 has **no I/O port space**: everything is memory-mapped, so the CPU decodes nothing and every device answers an address instead of a port. Its vectors sit at the top of memory — RESET at `FFFE`, IRQ at `FFF8`, SWI at `FFFA`, NMI at `FFFC`.

It is the processor of two machines here: the **SWTPC 6800** (`swtpc`) and the **MITS Altair 680b** (`altair680`). Both boot a ROM monitor the moment they start.

Every CPU board carries the same three properties: `clock_hz` (the crystal), `idle` (stands the processor down at a prompt), and the read-only `achieved_hz` (the speed it actually reached).

6809 — a Motorola 6809

The `6809` board is a Motorola MC6809 processor and its crystal. It has the same three properties as the `6800` board. No built-in machine uses it yet. Add it to a machine file, or type `BOARDS ADD 6809 cpu0`.

The 6809 has seven vectors at the top of memory: SWI3 at `FFF2`, SWI2 at `FFF4`, FIRQ at `FFF6`, IRQ at `FFF8`, SWI at `FFFA`, NMI at `FFFC` and RESET at `FFFE`. `SHOW BUS IRQ` shows all seven. The board takes IRQ from the bus. Its FIRQ input is not connected.

`DISASM` and `EDIT` use the 6809 instruction set when this board is the processor. On a 6809, `<nn` is a direct address and `>nnnn` is an extended address. `REGS` shows `A`, `B`, `X`, `Y`, `U`, `S`, `DP`, `PC` and the flags `E` `F` `H` `I` `N` `Z` `V` `C`. `D` is reachable by name.

The crystal is on the board — `clock_hz`

Because the crystal is soldered to the CPU card, `clock_hz` is the *board's* property, not the machine's. **`clock_hz = 0` is the default, and it means run flat out** — on a modern host, many times a real machine's speed.

```
SET cpu0 clock_hz=1000000
```

buys back a period ~ 1 MHz machine, and it is worth doing once. **What the guest sees is identical either way:** instructions cost the same cycles and a cassette loads in the same number of them, so the crystal buys period *feel*, not period *behaviour*.

That holds everywhere except at the edge of the machine. A guest counts instructions to measure time, so flat out it retires a "three-second" timeout in milliseconds of yours — which is why anything the guest times against the *outside* world (a transfer through a serial port) wants the real crystal. The troubleshooting chapter has the full story.

To see where the guest's own clock has got to, ask the machine:

```
swtpcsim> SHOW CLOCK
clock  (emulated time -- cycles since POWER, and what they are worth)

elapsed    1.103268 s    (2206536 cycles)
crystal    2000000 Hz    SET cpu0 clock_hz=N
pacing     free -- emulated seconds pass as fast as the host allows
```

Elapsed is counted in cycles since power and divided by the crystal above, so it is the guest's time and not yours — booting FLEX costs the same emulated second whether you ran it flat out or at 1 MHz. That is the number to measure a guest's own timeout against.

idle — the CPU stands down at a prompt

At a prompt a guest is only spinning on a serial status register waiting for a keystroke, and flat out that pins a core to accomplish nothing. **idle (on by default) stands the processor down while the guest is polling an empty console** — a pinned core becomes a few percent — and **the guest cannot tell**, because the moment a byte arrives the processor is back before the next poll.

Serial ports and consoles

The boards that carry a serial port — a console for SWTBUG, MON680 or FLEX to talk through.

mps — SWTPC MP-S serial interface

A **6850 ACIA** on an **SS-30 slot** — the board SWTBUG and MIKBUG expect for the console. It answers a four-address slot: control/status at the slot base and the receive/transmit data register at base+1, defaulting to `$8004` / `$8005`, the console slot. Unit `tty`. It is memory-mapped, and the ACIA's interrupt pulls the 6800 IRQ.

Only A0 reaches the chip

The 6850 has a single register-select line, `A0`, so it fills the **whole** four-address slot: `base+2` mirrors the base, `base+3` mirrors `base+1`. That mirror is load-bearing — SWTBUG's power-up probe compares the two to decide it is talking to an ACIA and master-resets it, and a board that decoded only two addresses would take the wrong path and emit garbage at boot. You do not have to think about it; it is here because the period software depends on it.

680io — Altair 680b onboard I/O

The 680b's built-in I/O: a **6850 ACIA console**, unit `tty`, at `F000 / F001`, plus the board's **configuration-strap read port** at `F002`, which the monitor reads to learn how the console is set up. Memory-mapped. This is the board `altair680` talks to, and `MON680` comes up on it out of the box.

680uio — Altair 680b Universal I/O

The 680b expansion board: a **second 6850 serial port** (unit `serial`) and a **6820 PIA** parallel port on one card, in an S9-relocatable window (default base `F000`: the serial port at `F006 / F007`, the PIA at `F008 – F00F`), plus fixed switch inputs at `F003` and a latch-free output at `F010 – F013`. The PIA is exposed as sections — `p1a / p1b`, and `p2a / p2b` when a second PIA is fitted with `pias=2` — each a line you `CONNECT`. Active-high.

Cassette

680kcacr — Altair 680b KCACR

The 680b's **audio-cassette interface**: a 1602-family UART recording the **Kansas City Standard** FSK, so a byte on the bus becomes an audible tone on a tape and back again. It is memory-mapped and **active-low**, with status/control at `F010` and the data register at `F011`. Unit `tape`.

It adds **software motor control** (control bit D7 starts the transport, D6 stops it) and **interrupt-driven transfer** (the D0/D1 enables pull the 6800 IRQ). Like every cassette it has a **position**, so it brings the verbs `WIND`, `REWIND` and `EXTRACT`: `WIND` puts the head at a time on the tape (`mm:ss`, or `START / END`), so a tape holding several programs one after another is reachable; `REWIND` is the common case of `WIND START`; and `EXTRACT` demodulates a mounted `.WAV` and writes each program it finds out as its own `.TAP` file. **The tapes chapter is the one to read**, and the `altair680` cassette example is the machine to run.

Floppy

dc4 — SWTPC DC-4 floppy controller

A **Western Digital WD179x** floppy controller on the SS-30 bus, up to four 5¼" drives (`drive0` – `drive3`), and **the board FLEX boots from**. It spans two slots: a drive-and-side select latch at `$8014` , and the WD179x registers at `$8018` – `$801B` (command/status, track, sector and data). Memory-mapped and DRQ-polled, running at 1 MHz.

FLEX 2.0 and 3.0 boot from it through SWTBUG's **D** command: mount a FLEX disk on `drive0` , type **D** , and the loader reads the boot sector and brings FLEX up to its `+++` prompt. The disks chapter covers the geometries it recognises (35- and 40-track single-sided, and double-sided) and how mounting works; the `swtpc` FLEX example in `examples/` is that machine with a disk beside it.

speed — instant or byte-timed

FLEX polls the controller's DRQ rather than timing it, so by default (`speed = full`) the board collapses seek and per-byte timing and a disk boots near-instantly. Set `speed = real` and it times each step and byte the way the hardware did — slower, and able to show a driver the Lost Data error it would get if it fell behind.

Working with the backplane at the prompt

Everything a machine file can do to a board, you can do by hand.

Command	
<code>BOARDS</code>	what is in the backplane
<code>SHOW BOARDS</code>	the board types you can add, one line each
<code>SHOW BOARD <type></code>	one type's description and its settings (add <code>UNITS</code> for just the units)
<code>BOARDS ADD <type> <id></code>	fit a board
<code>BOARDS REMOVE <id></code>	pull one out
<code>SHOW <id></code>	one installed board's settings, with the legal values
<code>SET <id> <key>=<value></code>	change one

```
swtpcsim> BOARDS ADD 680uio uio0
swtpcsim> SET uio0 base=F000
swtpcsim> SHOW uio0
```

The keys are the same keys. `SET cpu0 clock_hz=1000000` at the prompt and `clock_hz = 1000000` in a machine file are the same property reached two ways — there is no separate config schema, which is the whole reason the board reference at the back can be exhaustive.

And when you have the machine you want:

```
swtpcsim> CONFIG SAVE mine.toml
```

writes it out, and it round-trips.

Looking a board type over before you fit it

`SHOW BOARD <type>` reads the *catalog*. It builds one of that board, describes it, lists its settings, and throws it away — nothing is added to the backplane. A disk controller, for instance, ends by naming the units it carries:

```
swtpcsim> SHOW BOARD dc4
dc4  SWTPC DC-4 floppy disk controller: a WD179x (1 MHz) with up to four drives...
...
This board has units: drive
SHOW BOARD dc4 UNITS for their properties.
```

Add `UNITS` and you get just the units — each under a heading that names its **kind** and, after it, the **verb that fills it**:

```
swtpcsim> SHOW BOARD mps units

Unit 'tty' (serial, CONNECT)
connect  The endpoint on the other end of this line (CONNECT sets this)
...
```

A **serial** unit is an endpoint, so you `CONNECT` it (to `console`, a socket, a real port). A **disk**, **rom** or **tape** unit holds an image, so you `MOUNT` a file into it. A **cpu** unit is soldered on — neither — and shows only its kind. The heading tells you which verb a unit takes without your having to fit the board and find out.

Where a unit is filled from a *machine file* rather than by hand, `UNITS` shows the TOML table and its keys instead — `[[board.drive]]` for a disk controller, with its `mount`, `readonly` and `media` keys — so the file form is as discoverable as the prompt form.

Contention

Two boards decoding the same address is **contention**, and it is a real thing that real backplanes did. Both boards answer, both drive the bus, and what the processor reads is neither.

The simulator does not stop you. It does something more useful — **it tells you**:

```
swtpcsim> SHOW BUS CONTENTION
```

which names the address and names both boards. Fit two SS-30 boards at the same slot base and this is what you will see, and it is a great deal more helpful than a guest that has mysteriously gone mad.

Being able to build a machine that does not work is not a defect. **It is the point** — this is a bench for developing hardware, and hardware that cannot be wired up wrong cannot be wired up at all.

Disks

A disk on these machines is really three things: a **controller** board on the bus, some number of **drives** hanging off it, and a **disk image** sitting in one of those drives. The three are separate, and the manual keeps them separate, because the machine did.

This chapter is about the last two — the **image** and the **drive it goes in** — and the `MOUNT` workflow that puts one in the other.

The controller is a board. It goes in the machine file. The drives are part of the controller — the SWTPC **DC-4** addresses up to four of them. The disk goes in a drive, and you may put it there either way: name it in the machine file, or `MOUNT` it at the prompt. They do the same thing.

A disk is not a cassette, and the difference is real. A machine file *may* name the floppy that is in drive 0 — the FLEX example does exactly that — but it never names the tape in the recorder. A floppy drive is wired to its controller and the guest can tell what is in it. **A cassette is something a human puts in and presses PLAY on**, and it stays at the prompt where humans are. See the tapes chapter.

The controller

`swtpcsim`'s disk controller is the SWTPC **DC-4**, board type `dc4`, and this chapter's examples use it. Its hardware detail lives in the **Boards** chapter; here it is just the thing the drives hang off:

Board	What it is	Drives
<code>dc4</code>	SWTPC DC-4 — a WD179x floppy controller. The board FLEX boots from.	up to 4

The DC-4 is **memory-mapped**, not port-addressed: its drive-select latch and the WD179x registers answer addresses on the SS-30 bus. Drives are units on the board: `drive0`, `drive1`, and so on up to four.

Putting a disk in — `MOUNT`

```
MOUNT <id>[:<unit>] <file> [WP] [CREATE]
```

```
swtpcsim> MOUNT dc40:drive1 games.dsk
```

That is a floppy going into drive 1 of the controller called `dc40`. The socket was empty before; it isn't now.

The file has to be there already. A name that does not exist is not a new disk, it is a typo, and you are told so rather than handed a disk you did not mean to make:

```
swtpcsim> MOUNT dc40:drive1 my-scratch.dsk
dc40: 'my-scratch.dsk': no such file
dc40: to make a blank one, add CREATE: MOUNT dc40:drive1 my-scratch.dsk CREATE
```

CREATE is how you make one on purpose, and "Making a scratch disk" below is what to do with it once you have.

An **.imd** is converted on the way in

MOUNT on a file whose name ends **.imd** (an **ImageDisk** file, a format much archived floppy software is distributed in) does not mount the **.imd** itself. It **reads it, converts it to a raw sector image, writes that beside it as **foo.dsk**, and mounts the **.dsk**** — because a controller reads raw sectors, not ImageDisk's track records. The conversion is reported so you can check it against the disk you expected:

```
swtpcsim> MOUNT dc40:drive0 flex.imd
dc40: converted flex.imd -> flex.dsk
dc40:   IMD: FLEX 2.0 system disk
dc40:   102400 bytes
dc40:drive0: mounted flex.dsk
```

It **never overwrites a **.dsk** that is already there** — the same caution as **CREATE**. If **flex.dsk** already exists you are told to remove it or mount it directly, so a re-mount cannot silently discard edits you made to the converted disk. From then on the **.dsk** is the disk; the **.imd** is just where it came from.

WP write-protects the disk, and it does what a real diskette's write-protect notch did: the guest may read the disk, and a write is refused at the controller and never reaches the file on your host.

```
swtpcsim> MOUNT dc40:drive2 golden-master.dsk WP
```

R0 is accepted and means exactly the same thing. It is the right word for a ROM socket, which is read-only because of what it is — for a floppy, **WP** is the thing you are actually doing.

A protected disk says so wherever it is listed, so you never have to remember which one you did it to:

```
swtpcsim> SHOW MOUNTS
UNIT      KIND  HOLDS
dc40:drive2  disk  golden-master.dsk  (write-protected)
```

And if the **host** would not let us write the file — the permissions say no — the disk still mounts, protected, and says that it was not your idea:

```
dc40:drive2 disk master.dsk (write-protected -- THE HOST WON'T LET US WRITE IT; you did not ask for this)
```

That one is worth reading closely. You did not ask for the protection, so the guest is about to bounce every write off a disk you believe is writable.

And taking it out:

```
swtpcsim> UNMOUNT dc40:drive1  
dc40:drive1: unmounted (the drive is now empty)
```

The socket is empty again. The guest sees a drive with no disk in it, which is a thing a real drive could be.

Names, and what you may leave out

Board names are **case-blind everywhere** — `dc40`, `DC40` and `Dc40` are the same board.

Beyond that you may omit **what carries no information**:

- the **trailing index**, when only one board of that type is in the machine. One floppy controller means `dc4` finds `dc40`.
- the **unit**, when the board has only one thing you could possibly mount into. A cassette has one slot, so it needs no unit.

A floppy controller does not qualify for the second one. It has several drives — up to four on a DC-4 — and which drive you meant is real information, so you must say it. **Anything genuinely plural, you must name.**

...or put it in the machine file

`MOUNT` at the prompt is for the disk you are dealing with *now*. A disk that belongs to a machine belongs in the machine file, and that is how the shipped examples do it:

```

[[board]]
id = "dc40"                # no `type`: the controller is already there

[[board.drive]]
unit      = 0
mount     = "FLEX2-40.DSK" # relative to THIS FILE
# writeprotect = true      # refuse every write; your file cannot change
# create       = true      # make it blank if it isn't there -- CREATE, in a file
# media        = "flex40"  # what a blank one is; see "The geometry is probed" below

```

`readonly = true` says the same thing as `writeprotect` — both refuse every write; `MOUNT` takes `WP` for it, and `SHOW MOUNTS` prints `(write-protected)`.

The two forms do the same thing. The difference is only *when*: one is the drive as the machine ships, the other is you, at the prompt, changing your mind.

Note the path. It names the file lying **beside the machine file**, with no directory at all — which is what lets the whole folder be copied somewhere else and still boot. A path inside a machine file is resolved against **that file**; a path you type is resolved against **your shell**. The machines chapter has the rest of that rule.

...or fetch it over the network

Everything so far assumed the image is a file on your host. It need not be. Point `MOUNT` at a **TNFS server** — the network file system the FujiNet project speaks — and the disk is fetched across the network instead of read off your disk:

```

swtpcsim> MOUNT dc40:drive0 tnfs://fileserver/flex/games.dsk
swtpcsim> MOUNT dc40:drive1 tnfs://fileserver:16384/scratch.dsk WP

```

The form is `tnfs://<host>[:<port>]/<path>`. The port may be left off and defaults to **16384**, the TNFS port; the path is where the image lives on the server. A machine file may name one the same way — `mount = "tnfs://fileserver/flex/games.dsk"` on the drive — and because a `tnfs://` name carries its own server with it, it is **not** re-based against the machine file or your shell the way a bare filename is. It means the same thing wherever you write it.

What happens after that is what happens with any disk. The image is pulled in **once, at mount**, and from then on it behaves exactly like a local one: the guest reads and writes it, the geometry is probed from its size (below), `SHOW MOUNTS` lists it, and your changes are written back to the server when you `UNMOUNT` or the guest flushes. `WP` write-protects it just as it does a local disk, and if the **server** will not let us write, the disk still mounts, protected, and says the protection was not your idea.

A network can go away in the middle of a session, and a local disk cannot — so this one case is worth watching for. If the server stops accepting writes while you are using the disk, `swtpcsim` **tells you**: it prints a line saying it can no longer save changes to that mount, and that they are being held in memory only. The guest keeps running, and `swtpcsim` keeps trying; when the server comes back it says so and the held changes are saved. But until then, treat those changes as not yet safe — if you quit or the server never returns, what was written after the warning is lost.

The geometry is probed, not declared

You do not tell `swtpcsim` what kind of disk you just mounted. It **looks at the file's byte count** and works it out. The DC-4 recognises the FLEX formats:

Format	Tracks	Sides	Sectors	Bytes/sector	File size
<code>flex35</code>	35	1	10	256	89,600
<code>flex40</code>	40	1	10	256	102,400
<code>flexds</code>	40	2	10	256	204,800

A file of 102,400 bytes is a 40-track single-sided FLEX floppy. There is nothing else it could be. This is why the quick start never mentions a format: there was nothing to mention. FLEX numbers its sectors from 1, and its system information record lives on track 0 — the probe reads the size, and the driver takes it from there.

Only the 40-track image ships. The FLEX example carries a `flex40` disk; a `flex35` or a `flexds` image is one **you supply**. The board reads all three.

And a file that matches nothing at all

A blank disk is a file of no size, and a size of zero is not in the table. It mounts anyway, because refusing it would leave you no way to make a disk:

A file whose size matches no row is mounted UNFORMATTED, at the widest format the board can reach, and the guest's own formatter fills it in. The file **grows as it is written**, out to as far as the head can step. That is what makes `CREATE` and a period format program add up to a disk.

If you want a blank disk of a particular geometry — because you are about to format it with something that expects a certain number of tracks — say so with `media`:

```
[[board.drive]]
unit    = 1
mount   = "scratch.dsk"
create  = true
media   = "flex40"      # not "as far as the controller can step"
```

`media` is a machine-file key on the drive, not something you can set at the prompt. It also settles a truncated image, or a format you are inventing, where the byte count genuinely cannot decide.

Get back to the prompt before you stop

A mounted disk is mounted **read/write**, and every write goes through to the file on your host as it happens. A disk operating system buffers, though — a directory update or the tail of a file may still be in memory when your command returns — so the rule is simple and worth keeping:

Get back to FLEX's `+++` prompt before you `UNMOUNT`, copy, or stop the machine.

At the prompt the guest has finished writing, and the disk on your host is what you think it is. `^E` back to the monitor from `+++` is safe; `^E` one instant after a program says it has saved your file, and before the prompt returns, may catch the disk mid-write.

The disk is real, and there is no undo

Every write goes through to the file on your host, and `SNAPSHOT` will not save you: it captures the machine's *state*, not your host files, so the disk image is not in it. A guest cannot save your work onto a disk it is not allowed to write, so read/write is the only default that lets the machine be a machine — and the price of it is that you can destroy the example disk with one mistyped command.

Copy the folder before you experiment. It is self-contained and boots from anywhere. Use `WP` (or mount read-only in the machine file, as the shipped FLEX example does) when you want the guest to look and not touch.

Making a scratch disk

It is two steps, not two ways, and the second one is the guest's — which is exactly how it was in 1976.

CREATE gets you the blank medium; a formatter makes it a disk.

Step one, from the monitor. `CREATE` makes the file and puts it in the drive:

```
swtpcsim> MOUNT dc40:drive1 my-scratch.dsk CREATE
dc40:drive1: created my-scratch.dsk (empty)
dc40:drive1: mounted my-scratch.dsk
```

Zero bytes, and mounted — an unformatted disk, which is a thing you can hold in your hand and a thing FLEX will refuse to read. That is the correct state for a disk nobody has formatted yet.

Step two, from inside FLEX. Boot a system disk on drive 0, put the blank in another drive, and format it the way the period did — FLEX's own `NEWDISK` lays down the track structure and the system information record. When it finishes, the file on your host has grown from nothing to a formatted FLEX disk, and the drive is one FLEX will `CAT`, copy to, and put files on.

If you want a blank disk of a particular geometry rather than the widest the controller can step, say `media = "flex40"` (or `flex35`, `flexds`) on the drive — see "The geometry is probed" above.

Either way, remember which chapter you are in: get back to `+++` before you go looking at the file.

Looking at what is in the machine

`SHOW` on the controller lists its drives and what is in each one:

```
swtpcsim> SHOW dc40
```

`BOARDS` shows the backplane — every board, where it decodes, and who is fighting whom:

```
swtpcsim> BOARDS
```

Between them they answer nearly every "why is it not booting" question there is, and the first one is almost always *the disk is in the wrong drive*.

Tapes

Before the floppy, there was a **cassette recorder**. Not a special one — a home audio cassette deck from a department store, with a microphone jack and an earphone jack, and a card that turned bytes into a noise it could record and turned the noise back into bytes. On the Altair 680b that card is the **KCACR**, and it is in this simulator.

The chapter ends with the thing this is all for: loading a program off a cassette the way it was actually loaded — nothing on a disk, a loader in a small ROM, and a tape.

The KCACR

`680kcacr` is the Altair 680b's **audio-cassette interface**. Underneath it is a 1602-family UART with a **Kansas City Standard** FSK modem soldered to it: the guest reads and writes a serial register, and the modem on the other side turns the bits into tones. It is memory-mapped and active-low — status/control at `F010`, data at `F011`.

It has one unit: **tape**. There is one slot in a cassette recorder.

You cannot **CONNECT** it to anything

The KCACR takes no endpoint. It cannot be wired to your terminal, to a socket, or to a real serial port, and the reason is simple: **the line is soldered to the modem**. The signal goes to a cassette deck and nowhere else. **It is a cassette interface, not a serial port**, and the fact that it is built out of a serial part does not make it one. The serial chapter is about the boards that *do* take endpoints.

It can start and stop the tape, but not wind it

The KCACR has **software motor control**: a write to the control register with bit D7 set starts the transport and D6 stops it, so a guest program can run the recorder while it reads or writes. It can also transfer under interrupt (the D0/D1 enables pull the 6800 IRQ). What a motor line cannot do is say *which way* — so winding the tape to a position is still something you type, below.

Working a tape

The tape is not hardware

The tape is NOT in the machine file, deliberately.

A machine file describes the **hardware** — what boards are in the backplane, what they decode, how much memory is on the bus. Which cassette is sitting in the recorder is not hardware. It is a thing you did with your hands, this morning, and you can do a different thing with your hands this afternoon without unscrewing the lid.

You put the tape in, and you press **PLAY**. That is `MOUNT`, and it is a thing you type.

Putting a tape in

```
swtpcsim> MOUNT kc0:tape "kcacr-demo.tap"
```

The KCACR has one unit, so the unit carries no information, so you may drop it:

```
swtpcsim> MOUNT kc0 tape.bin
```

Names are case-blind. The rules are the ones in the disks chapter, and they are the same rules.

mode = play | record

```
swtpcsim> SET kc0:tape mode=record
```

play loads from the file; **record** saves to it. Which way the bytes are going is the whole of the setting.

The two are **mutually exclusive** — one tape, one head, one direction at a time — so `mode` is one setting with two values, and there is no third.

Where the head is — the tape counter

`SHOW MOUNTS`, and `SHOW <id>`, report where the head is sitting as a position on the tape:

```
swtpcsim> SHOW MOUNTS
kc0:tape  tape  demo.wav  00:15 / 01:28 (17%)  301/2048 bytes
```

The counter is a **time and a percentage** — minutes and seconds into the tape, and how far along it you are. For a `.WAV` that time is the recording's *own*: it counts the leader and any silent gaps between programs, exactly as a real cassette counter would. For a byte `.TAP`, which carries no audio, the time is estimated from the baud. The byte count is still there beside it.

WIND — move the head to a time

A tape unit brings a verb of its own:

```
WIND <id>:<unit> <mm:ss | START | END>
```

WI will do. It winds the head to a time on the tape — so a cassette holding several programs one after another is **reachable**: read the counter (or a manual) for where the next one begins, and wind there.

```
swtpcsim> WIND kc0:tape 2:05
kc0:tape: wound to 02:05 / 08:40 (24%) -- demo.wav (...)
```

The position is a time — `mm:ss`, or a bare number of seconds — or the words **START** and **END**. A time past the end lands at the end.

REWIND — wind to the start

REWIND `<id>:<unit>` (**REW**) is **WIND** ... **START**: the common case, kept as its own verb.

You need it to load the same tape twice. A tape that has been read is a tape whose head is at the end of the tape, and playing it again gets you silence. This surprises people exactly once.

```
swtpcsim> REW kc0:tape
```

Watching a load — the live counter

When a tape plays in real time (`rate = real`, below) the counter **ticks up on the console while it loads**, so you can watch a long tape's progress. It is on by default; turn it off for a machine whose guest is writing to the same terminal:

```
swtpcsim> MOUNT kc0:tape "tape.wav" counter=off
```

or **SET** `kc0:tape counter=off` at any time. Turning it off changes nothing about **SHOW** — the position is always there to ask for.

stop — halt the tape at a time

A multi-program tape runs one program straight into the next. The `stop` mark is your finger on the recorder's **STOP button at a counter mark**: set it to a time and the tape goes quiet there instead of running on. Set it at `MOUNT`, or with `SET` any time after:

```
swt pcsim> MOUNT kc0:tape "tape.wav" stop=2:05
swt pcsim> SET kc0:tape stop=2:05
```

Load program 1 and the line falls silent at 2:05 — the same quiet the end of the tape gives — so the loader stops there rather than reading into program 2. To carry on, move the mark forward (or clear it) and go again:

```
swt pcsim> SET kc0:tape stop=5:30      (the next boundary)
swt pcsim> SET kc0:tape stop=off       (play to the physical end)
```

It is `off` (play to the end) unless you set it. `SHOW` shows an armed mark as `stop @ 02:05`. It halts **playback only** — a recording writes straight through it — and it is independent of `WIND`.

rate = full | real

The cassette carries its own clock, and by default it runs flat out. `rate = full` — the default — hands the guest each byte the moment it is ready to read the next one, so a tape loads in about a second whatever speed the CPU is set to. This is almost always what you want: a program you are trying to run should not make you wait for a recorder that has been gone for forty years.

```
swt pcsim> SET kc0:tape rate=real
```

`rate = real` paces playback in **real wall-clock time** at the tape's baud, so a load takes as long as it took on the day. Set it when the *wait itself* is the point: a demo, a screen recording, the sound of the thing. It is playback only; recording takes as long as the guest drives it either way.

What this is *not* is the CPU clock. The tape's clock and the processor's are separate crystals on the real hardware, and they are separate here. A faster or slower CPU no longer drags the tape with it; `rate` is the only thing that governs how fast a tape plays.

Audio tapes — .WAV

Most surviving cassettes are not files of bytes. They are **audio**: somebody put a cassette in a deck, played it into a sound card, and saved a `.WAV`. You can mount one.

The recording is demodulated **once, when you mount it** — never while the machine is running — and from that moment everything above it, including `SHOW`'s byte count and `WIND / REWIND`, means exactly what it meant for a `.TAP`. The one thing the audio adds is a *real* clock: the tape counter reads the recording's own minutes and seconds, gaps and leader included, where a byte tape can only estimate them. The guest cannot tell.

```
swtpcsim> MOUNT kc0:tape "program.wav"
kc0:tape: mounted program.wav
program.wav: kcs300, 4439 bytes, 0 framing errors (100.0% of frames intact)
```

Read that first line. A mount always says what it found, and the framing-error count is the number that matters: a tape that decoded at 60% is noise, not a program, and you want to know that now rather than after the loader has crashed. A decode below 90% is refused outright.

But it is a framing rate, not a clean bill of health. The percentage counts frames whose start and stop bits landed where they belonged; it cannot tell you the eight bits between them are the ones that were recorded. A worn or poorly dubbed recording can keep its framing and still hand the loader wrong bytes. If a tape mounts well and the program still will not run, a worn recording is the likely answer.

What decides is the file's magic, never its name. A `.TAP` somebody renamed `.WAV` is still read as bytes, and a recording renamed `.TAP` is still demodulated.

extract — split a WAV into per-program `.TAP` files

A cassette WAV often holds several programs one after another, separated by a few seconds of silence.

extract writes each program out as its own `.TAP` — so you can keep, mount or load them one at a time instead of winding through the whole tape. Ask for it at `MOUNT`:

```
swtpcsim> MOUNT kc0:tape "games.wav" extract
kc0:tape: mounted games.wav
  games-1.tap  2048 bytes
  games-2.tap  3120 bytes
2 programs extracted
```

The files land **beside the WAV**, named from it: `games.wav` becomes `games-1.tap`, `games-2.tap`, ... with a 1-to-N index (a single-program tape is just `games.tap`, no index). `extract=<base>` names them yourself. It only **reads** the tape and **writes** the files — nothing in the machine changes.

The same thing has a verb, so you can split a WAV that is already in the deck without re-mounting:

```
swtpcsim> EXTRACT kc0:tape
```

Only a `.WAV` can be extracted — a `.TAP` is already the bytes, and has no gaps left to split on. The boundary is a second or more of silence, far longer than the gaps *inside* a program, so programs come apart cleanly and none is cut in half.

A board will refuse audio it could not really have heard

The KCACR's modem reads the **Kansas City Standard** (`kcs300`). The demodulator here measures the tones actually on the tape, so it *could* read another modulation perfectly well — but a real KCACR could not, and a tape whose tones are outside the modem's range is refused rather than decoded into bytes no real board could have produced:

```
swt pcsim> MOUNT kc0:tape "othertape.wav"
kc0: othertape.wav: this board's modem cannot hear that tape -- it carries ..., and this board reads kcs300
```

The frequencies in that message are a **measurement, not the tape's specification** — read it as "*not mine, and here is roughly what is there*" — and you go find a machine that reads it.

Recording back out to a `.WAV`

Put the recorder in `record`, and when the transport stops the tape is re-modulated and written back over the file — in the format and at the sample rate it was mounted with:

```
swt pcsim> SET kc0:tape mode=record
swt pcsim> RUN                      (the guest records)
swt pcsim> SET kc0:tape mode=play    (...and the WAV is written here)
```

You can also make a blank tape from scratch — `MOUNT ... CREATE`. A blank file has no recording to sniff, so its *name* decides what it becomes: a `.wav` name makes a blank **audio** tape that records a real, playable WAV; any other name makes a blank **byte** tape.

```
swt pcsim> MOUNT kc0:tape new.wav CREATE mode=record    (a fresh Kansas City audio cassette)
swt pcsim> MOUNT kc0:tape new.tap CREATE mode=record    (a fresh byte cassette)
```

The mount line tells you which you got, so there is no guessing. `mode=record` is there because the deck comes up in PLAY, one head and one direction: it is what lets the tape *record*, not what makes it audio — the name does that. Without `CREATE`, `MOUNT` needs a file that already exists, so a mistyped name is caught as a mistake rather than turned into a blank tape.

The stop is what writes the audio. `UNMOUNT` and any `WIND` (`REWIND` included) are stops too. The whole file is rewritten each time, not patched: the audio for a byte starts at an offset that depends on every byte before it, so there is no cheaper splice.

Time and level are the things that do not survive the round trip. A byte image holds no durations, so the leader a real transport needs has to be put back by whoever writes the audio — the `leader` and `trailer` properties (in seconds) do that. And the `level` property (percent of full scale) decides whether the tape reads back cleanly on real hardware: too hot overdrives a real deck's input and the tape reads its header and then fails. A `waveform` property chooses whether the tone is drawn square (what the real modem lays down) or sine (smoother, quieter). Their defaults are set for the board — `SHOW kc0` lists them, and the board reference at the back records the numbers.

Recording to a `.TAP` works as it always has, and is unaffected by any of this.

format, when you need to overrule the sniff

Each tape unit has a `format` property. `auto` is the default and is almost always right.

```
swtpcsim> SET kc0:tape format=raw
swtpcsim> SHOW kc0
```

(`SET` addresses the unit; `SHOW` addresses the **board**, and lists every unit on it with its properties underneath. There is no `SHOW <id>:<unit>.`)

Value	What it does
<code>auto</code>	Sniff for RIFF magic; demodulate a recording, read anything else as bytes
<code>raw</code>	Read the file's own bytes even if it is a WAV — how you inspect a tape that decodes badly
a modulation	Force the board's own — <code>kcs300</code> on the KCACR

It selects a *reading*; it never widens the hardware. A companion read-only `detected` property reports what the mounted tape turned out to be. `format` takes effect at the **next** `MOUNT`, because a tape is decoded once, when you put it in.

Loading a program off cassette — the ritual

This is the whole point of the chapter. On the Altair 680b, a small ROM holds the KCACR loader, and loading a program is: put the cassette in, then jump to the loader. The `altair680` cassette example is that machine, with a demo S-record tape already in the recorder.

1. Put the cassette in, and start the machine.

```
$ swtpcsim examples/altair680/altair680-kacr.toml
```

The example's machine file mounts `kcacr-demo.tap` in the recorder and boots MON680 to its `.` prompt. (You could type the `MOUNT` yourself instead — it gets no special powers.)

2. Run the loader.

```
.J FD00
```

MON680's `J` jumps to the KCACR loader/punch ROM at `FD00`. The loader starts the recorder reading, the tones come off the tape, and it deposits the program into memory — verifying every byte as it writes — then returns to the `.` prompt at the tape's terminating record.

3. Look at what landed.

```
.M 0200
```

The demo tape is a tiny S-record that writes four bytes at `0200`; `M` examines memory, and the byte the loader wrote is there. To load real 680b software instead, mount its cassette in place of the demo tape — the loader path is identical. The companion entry `.J FD74` **punches** memory back out, recording it to a tape in `record` mode.

Speed: the tape and the CPU are on separate clocks

The machine runs flat out by default, and so does the tape. The two are on separate clocks, as they were in the hardware: the processor's speed is `clock_hz` on the CPU board, the tape's is `rate` on the deck (above), and neither drags the other. `SET cpu0 clock_hz=1000000` buys back the period feel of the *processor*, while `SET kc0:tape rate=real` is the separate switch that makes the *load* take its real wall-clock time. In `rate = full` the guest cannot tell the difference at any CPU speed, because a polled loader only ever asks for the next byte and the byte is always ready.

Serial ports, sockets and telnet

A 1970s microcomputer had no screen of its own. What it had was a serial board, and whatever you chose to hang off it — a Teletype, a glass terminal, a modem, a paper tape reader. The board did not know or care. It moved characters.

`swtpcsim` keeps that arrangement exactly. **Any board that moves characters has one or more UNITS, and every unit can be CONNECTed to an ENDPOINT.** The unit is the socket on the back of the board. The endpoint is what you plugged into it.

```
swtpcsim> CONNECT uio0:serial socket:2323
swtpcsim> DISCONNECT uio0:serial
```

That is the whole of the interface. The interesting part is the endpoint grammar, and it is short enough to print in full.

The endpoints

This table is exhaustive. There are no others.

Endpoint	Is
console	the host terminal — your keyboard and your screen.
null	nowhere. Writes vanish. Reads never come.
loopback	itself. What the guest writes comes straight back as a read.
socket:PORT	LISTENS on that TCP port, as a raw pipe. Add <code>?banner</code> to greet each caller (see <i>The connect banner</i> , below).
socket:HOST:PORT	CALLS OUT to that host and that port, as a raw pipe.
telnet:PORT	LISTENS like <code>socket:PORT</code> , but speaks the Telnet protocol — so a <code>telnet</code> client behaves: no double echo, one key at a time. This is the telnet-in case for a person . Greets each caller; <code>? banner=off</code> stops that.
telnet:HOST:PORT	CALLS OUT like <code>socket:HOST:PORT</code> , taking the telnet client's part.
serial:DEVICE	a real serial port on this host.
in:PATH	a host file, read-only — a paper-tape reader . The file's bytes feed the board.
out:PATH	a host file — a paper-tape punch . Whatever the board sends is written to it.
in:PATH,out:PATH	both at once on one line: a reader and a punch, two files, two positions.
terminal	a terminal in a window the simulator draws itself — a built-in VT100 (or ADM-3A, VT52, H19). Present only in a build with a display. See <i>A terminal in its own window</i> , below.
printer:QUEUE	a real print queue on this host, write-only. Buffers the bytes into a job and prints it. Present only where the build found a host print system.
scripted	a terminal with a caller in place of a human. No tty need exist. It is what the MCP tools and the test suite type into; you are unlikely to type it yourself.

Any of these can be **tapped**: append `|FILE` to log the line to a hex file as it runs, or `|socket:PORT` to **mirror** it live — a second person `telnet`s in to watch the session and can type back onto the line to take over. Both are modifiers on an endpoint, not endpoints of their own — see *Tapping a line to a log file* and *Mirroring a line so a person can watch and take over*, below.

`null` is not an error

An unconnected unit is `null`, and **that is a legitimate state, not a fault**. A 6850 with no cable in it sits there with its transmit register permanently empty, forever ready, and a program that writes to it runs perfectly and talks to nobody. Reads never complete because nothing is sending.

Which is precisely what the real card does with no cable in it. A machine with a second serial port nobody plugged anything into is not a broken machine. `null` models the missing cable, and the guest is entitled to be fooled by it exactly as it would have been in 1977.

The colon is the whole distinction

`socket:2323` **listens**. `socket:localhost:2323` **calls out**. One colon, and it is the same convention every terminal program has used for forty years: a bare port is a port you own, a host and a port is a place you go. Nothing else about the endpoint changes.

Telnetting into the guest

Wire a unit to a listening `telnet:` port, and the guest has a serial port with a terminal on the end of it. That the terminal is your telnet client, several processes away, is not something the guest can discover.

```
swtpcsim> CONNECT uio0:serial telnet:2323
swtpcsim> RUN
```

Then, from another terminal on your machine:

```
$ telnet localhost 2323
Connected to swtpcsim 0.0.1 (uio0:serial) on port 2323
```

The guest is now talking to that window. Your first terminal still has the monitor and `^E` in it. This is how you give a machine two terminals, and it is how you drive a program that wants a console that is not the one you are sitting at.

socket: or telnet:

Both listen on a port (or call out to `HOST:PORT`), both raise carrier when a caller connects and drop it when they leave, and the guest cannot tell them apart. They differ in what they say to the far end:

	socket:	telnet:
Who is at the far end	another program or another machine	a person with <code>telnet</code> or <code>nc</code>
On the wire	the guest's bytes and nothing else	Telnet: it negotiates echo and one-key-at-a-time with the client
What the person sees	each key twice (their terminal echoes it, then the guest does), and Enter sent as a whole line	one echo, from the guest, and each key sent as it is pressed
Greeting	none, unless you add <code>?banner</code>	one line naming the machine, the board line and the port; <code>?banner=off</code> stops it

So: `telnet:` when a person types at the guest. `socket:` when a program is at the far end — one `swtpcsim` wired to another, a file transfer, a tool that speaks raw bytes, or a line you mirror.

A raw `socket:` shows its problem the moment a person uses it: a stock `telnet` client, left in its own defaults, echoes every key locally *and* the guest echoes it back, so each character appears twice, and Enter arrives as a whole line with its carriage return turned into a line feed. The fix is not on your keyboard. The two ends have to negotiate, and that is what the Telnet protocol is for.

The connect banner

A `telnet:PORT` line says hello to each person who connects, before the guest says anything — the `Connected to ...` line above. It names the build, the board line you reached and the port, so you know you have the right machine and the right line. Only the caller sees it. The guest does not, and nothing reaches the board. To turn it off, add `?banner=off`:

```
swtpcsim> CONNECT ui0:serial telnet:2323?banner=off
```

A `socket:PORT` line has no banner, because another machine is often at the far end, and it would take the banner as data. If a person is calling a raw socket and you want the greeting, add `?banner: CONNECT ui0:serial socket:2323?banner`. The banner is only for a port that listens. A line that calls out (`socket:HOST:PORT`, `telnet:HOST:PORT`) is the caller itself, so `?banner` there is refused.

A terminal in its own window

The telnet route above works, but it has moving parts you do not own: a telnet client must be installed, it must be pointed at the right port, and it mangles any byte it thinks is a telnet command — which breaks cursor sequences and file transfers, because to a serial line every byte is just data. And a modern host terminal emulates a modern terminal, not the ADM-3A or VT52 the 1970s software on the disk was written for.

`terminal` removes all of that. It is a terminal the simulator draws itself, in its own window:

```
swt pcsim> CONNECT mps0:tty terminal
swt pcsim> RUN
```

The guest's console is now that window; the monitor and `^E` stay in the one you launched from. There is nothing to install and nothing to connect — the window is up the moment the line is, on every platform. Because the terminal is the simulator's, the emulation is exactly the one you ask for, and it answers the reports (like `ESC[6n`) a period program expects.

Bare `terminal` is a VT100 in an 80×24 window. Options ride the connect string after a `?`, joined by `&`, the same as any other endpoint:

```
swt pcsim> CONNECT mps0:tty terminal?emulation=adm3a
swt pcsim> CONNECT mps0:tty terminal?emulation=vt52&size=80x24
swt pcsim> CONNECT mps0:tty "terminal?emulation=h19&size=132x24"
```

`emulation` is one of `vt100` (the default; `ansi` is the same engine), `adm3a` (the Lear Siegler ADM-3A, the classic 1970s serial terminal), `vt52`, or `h19` (the Heath/Zenith H19, a VT52 superset with an ANSI mode). `size` is `columns×rows` and defaults to `80x24`. `phosphor` is the tube colour — `green` (the default) or `amber`. `width` is the opening window width in pixels (a bare number, e.g. `width=1100`); left off, the window auto-sizes to about half the screen. The text is drawn in the real **DEC VT220** character set, and pairs with the period tube look — `[display] crt = true` (see [Configuring](#)), under which a `width` opens the window at exactly that size.

```
swt pcsim> CONNECT mps0:tty terminal?emulation=vt100&phosphor=amber&width=1100
```

A `terminal` needs a window, so it is available only in a build with a display. Ask for one in a build without, and it is refused cleanly at `CONNECT`, with the reason named; use `console` or a `socket:` there instead.

The built-in terminal has the same fold-the-bytes settings the console does — `strip7out`, `upper`, a CR/LF option and the rest — under `[terminal]`. They matter for a period monitor that sets bit 7; see *The*

built-in terminal has these too, later in this chapter.

Calling out

```
swt pcsim> CONNECT uio0:serial socket:bbs.example.com:23
```

The guest dials. As far as the software inside the machine is concerned it has a modem and the modem is connected; it will happily run a period terminal program over it.

A real serial port

```
swt pcsim> CONNECT uio0:serial serial:/dev/tty.usbserial-A600K1XY  
swt pcsim> CONNECT uio0:serial serial:COM3
```

The second form is Windows. The bytes go out of a real UART, down a real wire, into whatever you have on the other end.

If you get the device name wrong, it lists the ports that actually exist on your machine. It does not merely say "cannot open". A cable that appeared under a name one character off from the one you expected is ten minutes of somebody quietly deciding the simulator is broken, and the fix is to print the answer instead of the complaint.

What the board does to the wire

The host port is opened at 9600 8N1, and then **immediately re-programmed by the board** — because the board is the only thing in the system that knows what frame it is carrying. On the 6850-based serial boards here — the MP-S, the 680b onboard console, the 680b Universal I/O — the word format is a **register the guest writes**, not a property, and there **must not** be a property for it: a property would be a second place to say one thing, and the two would disagree the moment software touched the chip. So the frame on the wire is whatever the **guest** last programmed, and a guest that selects 7E1 reconfigures the cable to 7E1. Modem control lines — DCD, CTS, RTS — are wired through.

And give the machine its real crystal before you transfer a file

```
swt pcsim> SET cpu0 clock_hz=2000000
```

The moment the wire leaves the machine, the guest is talking to something that keeps time the way *you* do — and it does not: it counts instructions, so flat out it retires a "three-second" timeout in milliseconds

and decides your sender is dead. Flat out is right for a machine talking to itself; **a machine talking to you wants the crystal**. The troubleshooting chapter has the full story.

A paper-tape reader and punch: **in:** and **out:**

A serial or parallel line is where a **paper-tape station** lived, and `swt pcsim` gives you one out of two host files. The direction is the keyword, not a flag:

```
swt pcsim> CONNECT uio0:pla out:printout.txt      # a punch: capture what the board sends
swt pcsim> CONNECT uio0:pla in:reader.tap         # a reader: feed a file to the board
swt pcsim> CONNECT uio0:pla in:reader.tap,out:punch.tap # both, on one bidirectional line
```

in: is a **reader** — a byte *source*. It reads the file from the start, hands the bytes to the board one at a time, and when the file runs out the line simply goes **quiet**: no error, no end-of-file byte, exactly as a reader with no more tape sits idle. A file that is not there is a clean refusal at `CONNECT`, with the path named.

out: is a **punch** — a byte *sink*. Whatever the board sends is written to the file. It does **not** truncate: it overwrites forward from the start and extends past the old end, so a short run into a longer old file leaves the old tail behind — the same as spooling fresh tape over a reel that still had some on it. An absent file is created.

in: and **out:** are **separate files with separate positions**, so the combined form is two independent heads on one line — reading the tape cannot disturb what the punch has written, and vice versa. Both are **8-bit clean**: the bytes on the wire are the bytes in the file, control codes and all. Nothing reformats them. A relative path follows the usual rule: it resolves against the machine's directory, whether it is written in a machine configuration or you type it.

A reader with a speed

A real paper-tape reader has a rate, and a program that times its input cares. Pace an **in:** reader with `? cps=N` (characters per second) or `? baud=N` (a line rate at 10 bits per character):

```
swt pcsim> CONNECT uio0:pla in:tape.tap?cps=300    # a high-speed reader
swt pcsim> CONNECT uio0:pla in:tape.tap?cps=30     # a slow reader
```

Give exactly one of `cps` or `baud`, and a positive rate. With neither, the reader runs flat out — the generic default. The punch takes no options; it writes at the line's own speed.

Printing to a real printer

Where your build was made with host printing, a line can go to a real print queue instead of a host file:

```
swtpcsim> CONNECT uio0:pla printer:linewriter
```

Host printing is built on **macOS and Linux** today; the **Windows** builds do not have it yet, so on Windows `printer:` is absent and printing goes to a host file (`out:`) or a socket (`socket:`) instead. To see which your build has, connect to `printer:` with no name — it either lists the queues it can reach or tells you host printing is not in this build.

`printer:` is a **write-only** sink like `out:`, and just as un-printer-specific — any line can use it. The difference is what happens to the bytes: they are held in a buffer and then handed to the host print system as one **job**.

When does a job end? A printer has no "done" signal — a program prints and then simply stops. So `swtpcsim` decides the boundary for you, and you can tune it in the endpoint itself:

Option	Means	Default
<code>?idle=N</code>	end the job after N seconds with nothing more printed. <code>0</code> = never.	<code>5</code>
<code>?onff</code>	also end the job on a form feed (the page-eject character).	off
<code>?max=N</code>	end the job at N bytes , so a runaway program cannot fill memory.	a large number

```
swtpcsim> CONNECT uio0:pla printer:linewriter?idle=15
swtpcsim> CONNECT uio0:pla printer:linewriter?onff
swtpcsim> CONNECT uio0:pla "printer:Generic / Text Only?idle=0&onff"
```

Write a bare option (`?onff`) to turn it on; the common case never types `=1`. The boundaries combine — the first to fire ends the job — and an **empty buffer never prints**, so a form feed followed by silence does not leave a blank page behind. A job also goes out when you `DISCONNECT` the line, load another machine, or quit, so nothing you printed is ever left un-sent.

The queue must be one the host passes through **untouched** (a *raw* queue): a printer control language is not text, and a normal queue would try to reformat it. Creating that queue is a one-time step in your operating system's printer setup, outside `swtpcsim`. If a queue name contains spaces, quote the whole endpoint as shown above. Connect to `printer:` with no name and `swtpcsim` lists the queues it can see.

Like `out:`, a printer line is **8-bit clean** — the bytes the program sent are the bytes the printer gets.

Tapping a line to a log file (a poor man's protocol analyzer)

When you are trying to work out what a guest and the far end are actually saying to each other, you want to *see the bytes*. Append `|FILE` to any endpoint and `swtpcsim` writes every byte that crosses the line — both directions — to a text file as it runs, in hex and ASCII, with timestamps. The guest cannot tell the tap is there, and it never changes a byte, so it is safe on a binary transfer as well as on a terminal.

```
swtpcsim> CONNECT uio0:serial socket:2323|bbs.hex
```

Telnet in, drive the guest, and `bbs.hex` fills with the conversation:

```
# swtpcsim capture socket:2323 2026-07-29 14:03:11 fmt=dump
+0.001000 TX 0000 41 54 5A 0D ATZ.
+0.048213 RX 0000 0D 0A 4F 4B 0D 0A ..OK..
+9.100000 [DCD^]
+45.30000 [DTR_]
```

`TX` is what the guest sent, `RX` what came back; the offset counts bytes within a burst, and the right-hand column is the ASCII (a `.` for anything unprintable). Lines in `[...]` are **modem control-line** edges — carrier, DTR, RTS and the rest — logged as they change, with `^` for a rising edge and `_` for a falling one, so you can see the far end pick up and hang up.

The file is truncated each time you connect: a capture is a fresh trace, not an append. The whole tap is remembered — `SHOW` prints it and `CONFIG SAVE` writes it back — so a machine file can carry a line that is permanently traced.

Three layouts, and a few knobs

The tap's options ride the same `?key=value` grammar the other endpoints use, after the file name:

```
swtpcsim> CONNECT uio0:serial socket:2323|bbs.hex?fmt=cols
swtpcsim> CONNECT uio0:serial in:reader.tap?cps=300|trace.log?fmt=jsonl
```

- `fmt=dump` (the default) is the layout above: one hex row per line, strictly in time order — the easy one to read, and the easy one to `grep`.
- `fmt=cols` puts what the guest sent on the **left** and what it received on the **right**, so a request and its reply read down the page like a transcript.
- `fmt=jsonl` writes one JSON record per transfer — for feeding another program, diffing two runs, or importing into a spreadsheet.

The rest, all optional: `ts=elapsed` (the default — seconds since the first byte), `ts=wall` (the host clock), or `ts=none`; `width=N` bytes per hex row (default 16); `gap=MS` for how long a quiet line waits before a partial row is flushed (default 200 ms); and `pins=off` to leave the modem control-line edges out. Note that the second example taps a **paced paper-tape reader** — the tap composes with any endpoint, options and all.

Mirroring a line so a person can watch and take over

A tap writes to a file. A **mirror** writes to a *socket*, and it is a two-way wire: append `|socket:PORT` to any endpoint and a second person can `telnet localhost PORT` to watch the very session the machine is having — every character the guest prints — and **type back onto the line**, sharing it.

```
swtpcsim> CONNECT mps0:tty console|socket:2323
```

Now the guest talks to your terminal as before, and anyone who telnets to port 2323 sees the same output and can join in at the keyboard. It is the same idea as the tap — a modifier that composes with any endpoint — but where the tap only listens, the mirror also speaks. The everyday use is a console being driven by a program: a person telnets in, watches it work, and takes the keyboard when they want to.

There is no echo added by the mirror. What the watcher sees is exactly what the guest sends, and what the watcher types is input the guest reads — so if the guest echoes (a monitor, FLEX), the typed characters come back the ordinary way, and a password the guest does *not* echo stays unseen at the mirror too. One watcher at a time, the same as a serial line is one wire.

Add `?ro` to make it **watch-only** — a spectator who cannot touch the keyboard:

```
swtpcsim> CONNECT mps0:tty console|socket:2323?ro
```

The watcher never sets the pace. If a watcher's connection is slow, or they pause their terminal, the guest runs on regardless — a laggy watcher loses a little scrollback, never a byte of the guest's. Like the tap, the mirror is remembered: `SHOW` prints it and `CONFIG SAVE` writes it back.

A `CONNECT` it does not understand is an error

If `swtpcsim` cannot make sense of your endpoint, it **refuses, and tells you what it could have meant**. It never quietly falls back to `null`.

This matters more than it sounds like it does. A silent fallback gives you a machine that boots, runs, prints nothing, and hands you a dead terminal to debug — and you will debug the guest, and the board, and the

disk, before you think to doubt the thing you typed. A refusal is a worse morning for exactly two seconds. A dead terminal is a worse afternoon.

Exactly one unit may hold the console

The console is **your keyboard**, and there is one of it.

```
swtpcsim> CONNECT uio0:serial console
console taken from mps0:tty
uio0:serial: connected to console
```

Connecting a second unit to `console` **steals it, and says who it took it from**. It is not an error and it is not shared. Two boards reading one keyboard would each get roughly half the characters, in an order neither of them could predict, and the resulting machine would appear to be haunted.

To see who has it:

```
swtpcsim> SHOW CONSOLE
```

That also shows the transforms, which is the rest of this chapter.

The transform chain belongs to the console

This is the most important rule in the chapter, and it is worth stating twice before explaining it.

The `[console]` settings are the only thing in the simulator that alters a byte. Every serial LINE is 8-bit clean. There is no knob anywhere on any board that masks a bit.

They belong to the console, so they stop where the console does. Send a board's console unit down a `socket:` or a real `serial:` port and the far end gets the bytes as the guest wrote them — see *Where the transforms stop*, below, because it is the same rule and it surprises people.

Setting	Does
<code>upper</code>	folds what you type to upper case
<code>strip7in</code>	clears bit 7 of every character you type
<code>strip7out</code>	clears bit 7 of every character the guest prints
<code>crlf</code>	translates line endings
<code>echo</code>	echoes your keystrokes locally
<code>bell</code>	rings the terminal bell on <code>^G</code>
<code>bsdel</code>	folds backspace and delete together: <code>off</code> (default), <code>bs</code> (send BS for both), or <code>del</code> (send DEL for both)
<code>stop</code>	which control character is the STOP key (default <code>^E</code> ; <code>attn</code> is an accepted alias)
<code>base</code>	<code>hex</code> or <code>octal</code> — the base the monitor prints numbers in. Not a transform: it changes nothing about a byte crossing the console, only how a number is spelled back to you. The <i>Monitor</i> document has it.

Set them with `CONSOLE k=v`. (`SET CONSOLE k=v` is the same thing said longer.)

```
swtpcsim> CONSOLE strip7out=on
swtpcsim> CONSOLE upper=on crlf=off
swtpcsim> CONSOLE stop=1D
```

`stop=1D` moves the escape key from `^E` to `^]`. **It must be a control character** — a STOP key you can type by accident in the middle of a sentence is not an escape key, it is a trap.

Why it works this way: the high bit

Some period software sets **bit 7 of the last character of every message** — a string terminator, the way a routine knows where its text ends — and sends it. Run it with the transforms off and a word like `SIZE` comes out `SIZ` followed by a garbage character: the program is not broken. `E` is `45`; with bit 7 set it is `C5`, and your terminal prints whatever `C5` happens to mean to it.

The fix is `CONSOLE strip7out=on`, and the reason the fix lives *there* is the whole argument:

On the real machine, the board sent all eight bits. Nothing masked anything. The board put `C5` on the wire because that is the byte the program handed it. The Teletype on the other end simply **did not look at bit 7** — on a Model 33, that is the parity position, and the printing mechanism does not decode it. It printed `E` and threw the eighth bit on the floor.

Nothing was masked. Something on the far end did not look. **strip7out** is the terminal not looking. It is a property of the thing you are sitting at, and it belongs on the thing you are sitting at.

Where the transforms stop

Follow that argument one step further and it tells you something the first time it happens is alarming. If **strip7out** is your terminal not looking at bit 7, then the moment your terminal is **not** the thing on the far end, there is nothing there to do the not-looking:

```
swtpcsim> CONSOLE strip7out=on
swtpcsim> CONNECT mps0:tty socket:2323
mps0:tty: connected to socket:2323
```

Telnet in, and the high-bit garbage is back — exactly as though you had never set **strip7out**. The same is true of **upper**, **crlf**, **echo**, **bell** and **bsdel**, and it is true of a **serial:** port as well as a socket.

Nothing has been undone. The console settings are still on and still doing their job; the byte simply no longer goes through the console. It goes board → socket → your telnet client, and every hop on that path is 8-bit clean — which is the rule at the top of this section, not an exception to it. A transform that *did* travel down the cable would be a filter on a line, and the previous two sections are about why that is the one thing this simulator will not do.

So set the equivalent where it now belongs — on the terminal that is actually displaying the text. Every terminal emulator and telnet client has these, under its own names: strip parity or 7-bit display for **strip7out**, local echo for **echo**, newline or CR/LF handling for **crlf**. That is not a workaround; it is the same fix in the same place, one machine further out.

The built-in terminal has these too — in `[terminal]`

There is one terminal one machine further out that is the simulator's: the built-in `terminal` window from *A terminal in its own window*, above. It draws the text itself, so it is the simulator that must do the not-looking — and it gives you the same knobs the console has, under `[terminal]` instead of `[console]`:

Setting	Does
<code>upper</code>	folds what you type to upper case
<code>strip7in</code>	clears bit 7 of every character you type
<code>strip7out</code>	clears bit 7 of every character the guest prints
<code>cr</code>	<code>cr</code> (default, pass the guest's CR through) or <code>crlf</code> (add an LF after every CR)
<code>echo</code>	echoes your keystrokes locally, for half-duplex software
<code>bell</code>	passes <code>^G</code> through to the terminal (default on)
<code>bsdel</code>	folds backspace and delete together: <code>off</code> (default), <code>bs</code> , or <code>del</code>

Set them with `SET TERMINAL k=v`, read them back with `SHOW TERMINAL`, or put a `[terminal]` block in a machine file. Like `[console]`, it is one section for the machine — the window it applies to is whichever `terminal` line is open.

```
swtpcsim> CONNECT mps0:tty terminal
swtpcsim> SET TERMINAL strip7out=on
```

`strip7out` earns its keep here on an **even-parity monitor** — one that computes parity *into* bit 7 of every character, so it sends a carriage return as `8D`. To a console that is masked away; to the raw terminal window `8D` is not `0D`, so it prints as a glyph and the cursor never returns to the left — every line feeds without a carriage return. `strip7out=on` is the fix, exactly as it is for the high-bit prompt above. `cr=crlf` is the neighbouring tool, for the rarer guest that sends a bare CR and expects the terminal to supply the LF.

STOP is the other half of the same fact: it is intercepted at **your keyboard**, before any board is offered the byte, and it is never looked for on a socket or a serial line. A `05` arriving down a cable is a byte of somebody's protocol, and scanning a modem line for a key that exists only on the operator's terminal would corrupt data rather than help.

What *does* travel is the board's own line coding — baud, data bits, parity, stop bits — because that belongs to the board and not to you. That is the section after next.

Why not just strap the board to 7 bits

Because it would work, and then it would silently destroy your data.

Set a 7-bit mask on the board — or a filter on the line — and the high-bit prompt comes out clean. It also **silently corrupts every XMODEM transfer through that port**, because XMODEM sends binary, every byte of it matters, and bit 7 is a real bit in half of them. The file arrives. The checksums even pass on a bad packet often enough to be maddening. And the corruption is in the *plumbing*, which is the last place anyone looks.

A line may carry binary. A terminal is not a line. The transform belongs to the terminal because only the terminal knows it is displaying text.

data_bits and parity are real hardware, and are not this

A board genuinely does have `data_bits`, `stop_bits` and `parity`, and they are genuinely configurable, because a 6850 genuinely has those straps. They are a **FRAME** — they describe what physically travels down the wire, bit by bit, and on a real serial port they are what the far end must agree to or it will read garbage.

They are never a mask. `data_bits=7` is not "and the byte with `7F`". It is "put seven data bits in the frame", which is a statement about the wire, not about the byte the guest wrote. Do not reach for it to fix a prompt.

Worked examples

Complete sessions. **Every transcript below was captured from the program**, not typed out from memory — if it says the machine printed something, the machine printed it.

These are a few of the machines in `examples/`, gone through at length because each one teaches something about how the machine works rather than merely how to start it. **They are not the whole of what is there.** Every folder under `examples/` carries its own README — as Markdown and as a PDF beside it — saying what that machine is and what to type, so the place to see what you have is the directory itself, not a list in here.

1. FLEX 2.0 from a floppy

```
$ swtpcsim examples/flex/flex2-40.toml
```

```
startup> RESET
startup> RUN
[console -- ^E returns to the monitor]

$
```

You land at SWTBUG's `$` prompt — the SWTPC's ROM monitor, with a FLEX 2.0 disk sitting in drive 0. **There is no `BOOT` command.** On a real SWTPC you told SWTBUG to boot the disk with its `D` command, and that is what you do here:

```
$ D
FLEX 2.0
DATE (MM,DD,YY)? 1,15,80

+++
```

`D` reads FLEX's cold-loader off track 0 and jumps into it; FLEX signs on, asks the date, and drops to its `+++` prompt.

What is on the disk

```
+++CAT

DRIVE 0  ...
...file list...

...SECTORS LEFT
+++
```

`CAT` is FLEX's directory command. The disk carries the FLEX utilities and whatever else the image was built with; `+++` is the FLEX prompt, and the usual FLEX commands work from it.

Out, and back in

`^E` stops the machine and gives you the monitor. Nothing is lost — a bare `RUN` resumes at the instruction it was about to execute.

```
+++
STOP -- the machine is still at ADEE. RUN resumes.
H0I1N0Z1V0C0 A=0D B=00 X=8004 SP=A03D PC=ADEE  ...
swtpcsim>
```

Look at the machine while it sits there, then hand the keyboard back with `RUN`:

```
swtpcsim> BOARDS
swtpcsim> SHOW dc40
swtpcsim> DUMP A000
swtpcsim> RUN
```

Your FLEX prompt is exactly where you left it.

The disk is read-only, and that is on purpose

The example mounts drive 0 with `writeprotect = true`, so looking around FLEX can never dirty the image — a fresh clone stays a fresh clone, and there is no undo to need. To save files back (FLEX's `SAVE`, `NEWDISK`, a file copy), drop the `writeprotect` line from the drive in the `.toml`, or copy the `.DSK` somewhere writable first.

The board figured out the disk's shape by itself

Nothing in the machine file says how many tracks or sides the disk has. The DC-4 board probes the geometry from the image's **size** on mount — 100K here is a 40-track single-sided FLEX disk. Drop a differently-sized FLEX image beside a matching `.toml` and it boots the same way; the folder holds five sibling machine files for the other geometries FLEX shipped in.

2. MON680 on a bare Altair 680b

```
$ swtpcsim altair680
```

```
startup> RESET
startup> RUN
[console -- ^E returns to the monitor]
.
```

The `altair680` machine is a MITS Altair 680b: a 6800 with 1K of RAM and its onboard console — a single 6850 ACIA — and nothing else. It comes up straight into **MON680**, the 680b's ROM monitor, at a `.` prompt. There is no disk and no operating system; the monitor is the machine's software, and it is how you examine memory, deposit bytes, and run a program.

M examines and changes memory a byte at a time:

```
.M 0000 00 01
.M 0001 00 02
```

Type an address after **M** and the monitor prints the byte there and waits; type a new value to change it, or Return to move on. That — plus **G** to go and the register display — is the whole of driving a 680b, because in 1976 the monitor was the whole of the software you were given.

^E takes the keyboard back to the `swtpcsim>` monitor at any point, and **RUN** resumes:

```
.
STOP -- the machine is still at FF85. RUN resumes.
H0I1N1Z0V1C0 A=51 B=E0 X=0000 SP=00EE PC=FF85 BSR FF24
swtpcsim>
```

3. A cassette on the 680b's KCACR

```
$ swtpcsim examples/altair680/altair680-kcacr.toml
```

This is the same 680b with a **KCACR** audio-cassette interface added — the 680b's tape board, a UART recording **Kansas City Standard** FSK, with a loader/punch PROM at `FD00`. A cassette (`kcacr-demo.tap`) is already in the recorder: a tiny Motorola S-record tape that deposits four bytes at `0200`. From the `.` prompt:

```
.J FD00  
.M 0200 86
```

1. `J FD00` runs the loader PROM. It reads the tape's S-records into memory and returns to the monitor at the terminating `S9` record. (On a bad tape it prints one letter and stops: `C` = checksum/non-hex, `M` = memory error.)
2. `M 0200` examines `0200` and reads back `86`, the first byte the tape carried — proof the tape was read, byte by byte, the way a real KCACR read it.

The tape is not in the machine file

A machine file describes **hardware** — the processor, the console, the cassette *interface*. Which cassette is in the recorder is not hardware, so it is not in the file: you `MOUNT` it, the same verb an operator used to press PLAY.

```
swtpcsim> MOUNT kc0:tape "kcacr-demo.tap"  
swtpcsim> REWIND kc0:tape
```

`REWIND` is a verb the **cassette board brings with it** — it exists only because there is a KCACR in the machine. You need it to load the same tape twice, for exactly the reason you would have needed it in 1976.

It loaded in an instant, and a real one took time

The machine runs **flat out** by default, so a tape that took a real 680b the better part of a minute comes off at once. `SET kc0:tape rate=real` plays it at the true Kansas City speed if you want period *feel* — what the guest sees is identical either way. The `680kcacr` reads and writes Kansas City modulation only.

Where to go next

- The examples this chapter did not walk through — `examples/`, and the README in each folder.
- The other boards on the 680b — `examples/altair680/` also has a Universal I/O board example.
- Telnet into the guest, or wire it to a real serial port — the serial chapter.
- Look at the bus while it runs — the *Debugger* document.

The MCP server

```
$ swtpcsim --mcp
```

That runs `swtpcsim` as an **MCP (Model Context Protocol) server** on stdin and stdout, so that an AI assistant — Claude, or anything else that speaks MCP — can drive the machine through **typed, structured tools** instead of screen-scraping a text terminal.

It is not a wrapper, and it is not a second model of the world. **The MCP server runs on the same machine object as the monitor.** What the tools see is exactly what `SHOW` sees, because it is the same machine, answering the same questions, through a different door.

What the tools do

Enough to operate the machine:

- List the board types available, and every property each one has — **with its type, its default and its legal range.**
- List the boards actually in the machine.
- Get and set any property on any board.
- Add a board; mount a disk or a tape into it; wire a serial line to an endpoint.
- Examine, deposit, fill, search, save and disassemble memory.
- Run the machine, step it, set breakpoints, and read the bus flight recorder.
- Snapshot the whole machine's state and restore it.

The five you will see named are `board_types`, `board_list`, `board_get`, `board_set` and `board_add`. The rest follow the monitor's own vocabulary.

Driving a running guest

Building a machine is half of it; the other half is **operating one that is running** — typing at its console and reading what it prints. Four tools do that, and they are what let an assistant boot FLEX, run a program, and talk to it over a serial port entirely through MCP:

- **run** — advance the guest a bounded slice and return what it printed. It is the expect loop in one call: pass `input` to type a line, `until` to stop when a string (a prompt like `+++`) appears, `from` to set the PC first (booting is `from` the ROM monitor's reset entry). **It also stops on its own when the guest reaches a prompt** — spinning on the console with nothing to say — so you get control back without guessing a timeout. Every stop says why in `stopped: match, idle, timeout, steps, halt, breakpoint, unclaimed` (under `SET BUS UNCLAIMED=HALT`), `tape-stop` (a `BREAK TAPE`

STOP), or interrupted (see below). A SET BUS UNCLAIMED=WARN line, or any other bus or board message from the run, comes back in warnings . JSON has no hex, so from is a decimal number (65496 for FFD8); a string such as "0xFFD8" is refused with the number to send.

- **send** — type at the console without running (then run to let it be read).
- **recv** — drain what the guest has printed since you last looked, without running.
- **regs** — the CPU registers right now.
- **status** — a guaranteed-non-blocking check: whether the server is currently busy on ANY call (not just run), plus the CPU board id and the last run 's step count/PC. It never queues behind anything, including a run that never ends — see "Stopping a run that will not end" below for why that matters and what its fields mean when nothing is running.

The shape of a session is therefore: run {from: 0xE0D0, until: "\$"} to reach the SWTBUG monitor and run {input: "D", until: "+++"} to boot FLEX, then run {input: "CAT\r", until: "+++"} per command, reading the reply each time. A run **never blocks** — it advances the guest for at most timeout_ms (default 2000, maximum 600000) and returns — so a tools/call always comes back, unlike a bare RUN through the monitor tool, which under a pipe waits on a stdin that is the JSON-RPC channel itself.

That budget is a ceiling and not a wait. The call ends as soon as until matches or the guest reaches a prompt, so asking for more time than the work needs costs nothing: a job that takes fifty seconds under a budget of two minutes returns after fifty seconds. Set the budget to the longest you are willing to wait, not to what you expect, and let until end the call.

What you type goes to the guest byte for byte, control characters included, and every line in the machine is 8-bit clean. A control byte is written as a JSON \uXXXX escape: \u0003 is ^C, \u001b is ESC. So send {text: "\u001b"} pauses a FLEX listing exactly as the ESC key would. Note that \x03 is **not** JSON — JSON has no \x escape — and it arrives at the guest as the three ordinary characters x03 rather than as a control byte.

By default the guest runs flat out, which is what you want for booting and for driving a prompt. But when a real device is on a serial line and you have set a clock speed with SET cpu0 clock_hz=... , run paces the guest to that clock, so a reply the device sends a fraction of a second later lands while the guest is still waiting for it.

timeout_ms is a hard wall-clock ceiling, full stop — traffic on a live wire does not extend it. A boot loader that reads its whole system image in over a serial disk is not cut off mid-block; it is bounded the same way any other call is: give it a timeout_ms as long as the worst case takes (up to 600000 ms), and let it return early on until or a prompt the moment it finishes, exactly as a fast call does. A call that hits timeout_ms mid-transfer returns stopped: "timeout" with whatever it has read so far — a normal result to loop run on, not a failure, and regs / mem_dump can confirm a destination pointer is still climbing while you do.

Stopping a `run` that will not end

A `run` ends by itself at `timeout_ms`, but you may not want to wait that long. There are two ways to stop it early, and both make the `run` in progress stop at once and return `stopped: "interrupted"` with what the guest printed so far:

- **Cancel the request.** Send the standard MCP `notifications/cancelled` message naming the request id of the `run`. The server keeps reading its input while a `run` is going, so the cancel is seen straight away. A cancel that names some other request, or one that arrives after the `run` has returned, is ignored, and it never carries over to the next call. Other requests sent during a `run` are queued and answered in order once it returns — except `status`, which is the one call that is never queued: poll it to check whether a `run` you are considering cancelling is actually still alive, or already back to idle.
- **Send the process a `^C`.** Press it in the terminal that started the server, or run `kill -INT` on its process ID.

The machine is left exactly as it was, so you can look at it and carry on with another `run`. A `^C` that arrives while no `run` is in progress does nothing to the guest, and a new `run` always starts clean.

This changes what `^C` does to an `--mcp` server you started by hand: the first `^C` is caught, not fatal. If you press `^C` again before the server has reported the first one, the second one ends the server as `^C` normally would. A server started in the background, or with `nohup`, ignores `^C` altogether, as any such program does.

`status`'s `pc / steps` are only ever as fresh as the last `run` — a `step` or a `monitor` command moves the real PC without updating them, and `steps` resets to zero on the next `run`, so it is not monotonic across runs. `generation` is: it climbs on every publish, so it is the field to watch for "still advancing" versus "stuck on the same slice."

Under `--mcp` the console line is quietly re-seated onto an in-memory terminal the server owns (there is no host keyboard behind a pipe), which is what `send / run / recv` read and write. Everything else on the machine — a second serial board wired to a real port, a socket — keeps running and is serviced on every `run` slice, so a program shuttling bytes between the console and a modem port works exactly as it would at a real terminal.

Watching over your shoulder — `--mirror`

Add `--mirror socket:PORT` alongside `--mcp` and a person can `telnet localhost PORT` to **watch the very session the assistant is driving** — every character the guest prints — and **type back onto the line to take over**, sharing the console:

```
$ swtpcsim swtpc --mcp --mirror socket:2323
```

It wraps the assistant's console in the same mirror the monitor offers (`<endpoint>|socket: PORT`, see the *Serial lines* chapter). The assistant keeps driving through `run / send / recv` exactly as before — the mirror is transparent to it — while whatever it types and whatever the guest prints also crosses the socket to the watcher. Add `?ro` to make it watch-only — **quote it on the command line**, because `?` is a shell wildcard:

```
$ swtpcsim swtpc --mcp --mirror 'socket:2323?ro'
```

Without the quotes the shell tries to match `socket:2323?ro` as a filename and fails (zsh reports `no matches found`) before `swtpcsim` ever sees it. One watcher at a time.

The watcher never sets the pace, and one thing follows from that: the guest only advances **during a run**, so a character the watcher types between runs waits on the line and is read on the next `run` — the same as `send` staging input for the next `run`. While a `run` is in flight the two share the console live.

Debugging and inspecting

The monitor's debugger is here too, structured. `step` advances a set number of instructions and hands back the register file and where the CPU came to rest; `breakpoints` lists, adds and removes the same breakpoints `BREAK` sets — and because they are the machine's breakpoints, a `run` or a `step` stops when one fires. `disasm` decodes memory through a non-invasive read, so it works on a ROM and even with no processor running. `bus_trace` returns the always-on flight recorder — the last cycles every board saw, with who drove and who answered — and `bus_irq` reports the interrupt lines the way `bus_map` reports the decode. `snapshot` and `restore` save the whole machine's state and read it back into a machine built the same way.

None of these block, and none of them need the console: they are questions about the machine, answered the same way `SHOW` answers them. When a typed tool does not reach a corner you need — a conditional breakpoint, an octal listing — the `monitor` tool runs any monitor command and returns its text.

The schemas describe themselves

Every tool's schema comes off the same reflection layer as the TOML keys and the `SET / SHOW` commands. There is one description of what a board is and what it can be asked, and the machine file parser, the monitor, and the MCP server all read it.

The consequence is the point: **a board added tomorrow is drivable by an assistant the day it lands, with no new code.** Nobody writes an MCP tool for the new board. The board declares its properties, as it must anyway to be configurable at all, and the tool schema is that declaration.

So there is no tool reference in this manual. Start the server and ask it what it has — `tools/list` returns every tool the server exposes, and `board_types` every board type; their answers are authoritative in a way a printed list could never be.

Configuring an assistant to use it

MCP clients differ, but they all want the same two things: a command to run, and the fact that it speaks over stdio. The command is `swt pcsim <machine> --mcp`, and it does. Register it **once** and from then on you talk to the assistant, not to the server.

Claude Code (the command line) takes it as one command — everything after `--` is what it will run, and it is best run from the directory you want the machine's files to resolve against:

```
claude mcp add swt pcsim -- swt pcsim <machine> --mcp
claude mcp list                                # confirm it registered and is reachable
```

Claude Desktop, or any client that reads a JSON config, wants an `mcpServers` entry naming the command and its arguments (a `cwd` sets the working directory, since a desktop app has no shell to inherit one from):

```
{
  "mcpServers": {
    "swt pcsim": { "command": "swt pcsim", "args": ["<machine>", "--mcp"] }
  }
}
```

The `<machine>` is a built-in name or a machine file, exactly as on the command line. If `swt pcsim` is not on your `PATH`, give its full path as the command.

You are not limited to one. Register several machines under different names (`swt pc-flex`, `altair680`) and an assistant sees them all at once; `claude mcp add`'s *scope* decides whether a server is tied to the one

project directory you added it from (the default), travels with a folder as a `.mcp.json` (`--scope project`), or is available everywhere (`--scope user`). One thing to know for file exchange: the sandbox is the server's working directory. The `claude` command line sets that to wherever you started it, so a relative machine path and the sandbox line up with the folder you are in; the desktop app currently starts the server in your home directory instead, so there give the machine an absolute path.

`DRIVING-WITH-AI.md`, in this package, is the briefing written for the assistant itself — drop it in a working directory and the assistant has the recipes for booting, building and debugging over these tools. An example folder such as `examples/flex/` is a ready-made such directory — a machine and its disk together — for an assistant to drive over MCP.

Starting a project of your own

An example folder such as `examples/flex/` is a ready-made round trip; once you want to write your own software the same shape is how to begin. **Make a folder of your own, put a machine in it, and point the assistant at that folder — not at the copy that came in the package.**

Put the machine and its images at the root of the folder. Copy a machine that boots the system you want to build on — an `examples/flex/` folder is a good starting point, disk and machine file together — into a new folder, and work there. Register the server from inside that folder (as in the section above), so the machine file and the sandbox both resolve there. The guest writes to the disk image as it runs and the assistant leaves build files beside it, so keeping the shipped copy untouched means you always have a clean one to start over from. Make that a real baseline: keep a spare copy of just the machine and its fresh disk, so "start over" restores something known rather than whatever the guest last wrote.

Give it the local truth to read. Drop `DRIVING-WITH-AI.md` in the folder, and keep a second folder — call it `Reference` — for anything else you want the assistant to lean on: the parts of this manual that matter to your project, and any source material of your own, converted to plain Markdown, which is the form an assistant ingests most reliably. These copies go stale the moment `swtpcsim` updates, and the assistant cannot tell an old copy from a current one, so refresh them when you update `swtpcsim`.

Keep a hand on it — it will reach for what it already knows. An assistant does not read your machine's files and then do as they say. It predicts its next move from everything in front of it, weighed against everything it learned in training — and what it learned is vast, while a line in a local file is a single faint signal it saw a moment ago. So when the job looks like a common one — boot FLEX, assemble a file, copy something onto a disk — the move it reaches for first is the one it has seen a thousand times, even when the files in the folder say otherwise. A plain example: asked to get a program onto a disk, it will reach for a host tool that does not understand the DC-4's FLEX disks at all — when the way that works is to build inside the machine itself, exactly as `DRIVING-WITH-AI.md` lays out. The pull toward the familiar answer gets stronger the longer a session runs, and once it starts down that road it tends to talk itself further along rather than stop and re-read.

So watch what it actually sends and reads back — the `--mirror` socket above is there for exactly this — rather than trusting its own summary of what it did. Correct it at the first wrong step, before it builds on that step, and point it at the file you mean and tell it to use that and only that. Be most wary when its answer looks the most standard: that is the moment a habit has most likely overridden something particular about your machine.

Make starting and stopping a ritual. A short instruction you give every session pays for itself. At the end — "clean up" — have it close any processes still running, write down what you learned, and save where you are in a note in the folder. At the start, have it read that note and the `Reference` folder, boot the machine, and tell you where you left off. The note in the folder is what makes a session resumable; without it the assistant reconstructs the state from scratch each time, and reconstructs it wrong.

Troubleshooting

Most of what goes wrong here is not a fault. It is the machine doing exactly what a real one did, in a way nobody warned you about. This chapter is the warning, after the fact.

^C does not stop the machine

It is not supposed to.

^C belongs to the guest. FLEX reads it, and so do plenty of programs — it is a common way to interrupt. A stop key the guest also wants is a stop key the guest will eat, and then you are trapped inside your own simulator.

^E is the stop key. It stops the CPU and hands you the monitor; the host intercepts it before the guest is ever offered the byte, and **no program running inside the machine can disable it, ignore it, or take it from you.**

If a guest genuinely needs **^E** for something of its own, move STOP out of its way:

```
swtpcsim> CONSOLE stop=1D
```

That makes it **^J**. It must be a control character.

I pressed STOP and now the machine seems stuck

It is not stuck. It is **stopped**, which is what STOP is for, and nothing has been lost.

STOP halts the processor and hands you the monitor. It is not RESET and not POWER: the registers, the memory and the disk are precisely as the CPU left them, and the monitor told you where to pick it up when it printed "*still at ...*".

```
swtpcsim> RUN
```

A bare **RUN** — no address — resumes at that exact instruction, and your prompt (SWTBUG's **\$**, FLEX's **+++**, MON680's **.**) is where you left it. (**RUN <addr>** is a different thing: it loads the PC first, so it *restarts* rather than resumes.)

And while you are stopped, you can look: **REGS**, **EXAMINE**, **DUMP**, **DISASM** and **STEP** all work at this prompt, on a machine that is not moving under you. See the *Debugger* document. **BREAK** is for stopping at a place you cannot reach by hand.

My file was not written / the disk lost my changes

The shipped disk is **read-only**.

The FLEX example mounts its drive with `wriprotect = true`, so looking around FLEX can never dirty the image — which is what you want until you mean to save. But it also means FLEX cannot write to it: a `SAVE` or a file copy is refused at the controller. To keep your changes, drop the `wriprotect` line from the drive in the `.toml`, or copy the disk somewhere writable first.

The disk image changed and I wanted it not to

There is no undo once a drive is writable. **A read/write image is mounted like a real machine** — a guest that cannot write its disk cannot save your work.

Two answers, and take one *before* you experiment:

```
swtpcsim> MOUNT dc40:drive0 FLEX2-40.DSK R0
```

`R0` refuses every write at the controller, so your file is safe whatever the guest does. It is for a disk you mean to **read**: the guest is never told the disk is protected — the controller has no bit that says so — so a program that sets out to write to an `R0` disk will not fail gracefully. The disks chapter explains why.

or copy the folder. It is a directory with a machine file and an image in it, and it boots from anywhere:

```
$ cp -R examples/flex my-flex
```

`MOUNT` says "no such file" and the file is right there

Look at *which machine you are running*.

A relative path resolves against the machine's directory — the folder the machine file was loaded from — not the folder you launched from. If you `cd` somewhere with a disk, start a machine from elsewhere, and type `MOUNT dc40:drive0 that.dsk` expecting your current folder, it is looked for beside the *machine* instead, and that is the directory you were not thinking about. `SHOW PATHS` prints the one it uses.

This is deliberate, and it is the reason an example folder boots from anywhere: the machine file says `FLEX2-40.DSK` and means *the image next to me*, so you can move the folder, or run it from three levels up,

and it still finds its disk — and a name you type finds that same folder, so the disk the machine came with and the disk you mount by hand live in one place, not two.

Every prompt ends in a garbage character

Some period software **sets bit 7 of the last character of every message** as a string terminator, and your terminal prints the result — so `SIZE?` comes out `SIZ?` with a garbage character where the `E` should be. The `E` is there; it arrived as `C5` instead of `45`.

```
swtpcsim> CONSOLE strip7out=on
```

The real Teletype ignored bit 7 and printed the `E`. `strip7out` is your terminal doing the same. The serial chapter explains why the fix belongs on the console and **never** on the board.

Nothing appears when I type / the guest does not see my keys

Ask who has the keyboard.

```
swtpcsim> SHOW CONSOLE
```

Exactly one unit may hold the console. If the console is on a unit the guest is not reading, you are typing into a board nobody is listening to. Connect the right one — and note that connecting a second unit to `console` *steals* it from the first, and says so.

The machine runs impossibly fast — a cassette loads in one second

It is running **flat out**, which is the default. `clock_hz = 0` means "no divisor, go".

```
swtpcsim> SET cpu0 clock_hz=1000000
```

That is the real 1 MHz crystal, and it will give you the real cassette-load time. Both behaviours are correct; one of them is just a much longer lunch.

A file transfer keeps timing out — XMODEM, a serial upload

Slow the machine down. Transfers with the outside world want the real crystal.

```
swtpcsim> SET cpu0 clock_hz=1000000
```

Here is why, because it is worth understanding once and it explains a whole class of symptom.

A guest program has no clock. It counts instructions. A period transfer program times a second the way every program of the era did — by spinning in a loop and counting the trips — and that arithmetic is *only* a second at the real crystal speed. It waits a few of them for a block header. Run the machine flat out and those seconds — seconds of **cycles** — are retired by your host in a few tens of **milliseconds**, while the sending program on the other end of the wire is still living in seconds of the ordinary kind. The transfer concludes the sender is dead, NAKs, purges the line, and gives up. Nothing is broken. The two ends are simply no longer using the same clock.

And that is the boundary, exactly: timing inside the machine is consistent at any speed, because everything in there counts the same cycles — which is why a cassette loads correctly flat out, the KCACR and the tape agreeing with each other at whatever speed the pair of them run. A transfer to your host has **one end inside the machine and one end outside it**, and only the crystal makes those two agree.

So: flat out for everything the machine does to itself. The real crystal for anything it does with you. Set it back afterwards if you like the speed.

Two boards are fighting over an address

```
swtpcsim> SHOW BUS CONTENTION
```

names them. To see the whole map of who decodes what:

```
swtpcsim> SHOW BUS MAP
```

On real hardware this was two boards strapped to the same address and a bus you could not trust. Here it is a list.

A read from an address returns **FF** and I expected something

Nothing decodes that address. The bus floats high when no board is driving it, and **FF** is what a floating bus reads. It is not an error, and the machine will not tell you — a real one did not either.

To find out who *would* have answered, without running a cycle:

```
swtpcsim> WHO F400
```

And if the symptom is a **hang** — a machine you assembled yourself starts, prints nothing, and never comes back — the cause is often exactly this: it is polling an address for a board that is not there, reading

FF forever. Arm the bus to say so:

```
swt pcsim> SET BUS UNCLAIMED=WARN  
swt pcsim> RUN
```

WARN prints the address and the PC that reached for it and keeps running; **HALT** stops the machine right there, so **SHOW REG** and a **DUMP** show it exactly as it wedged. It names each absent address once per run. The default is **SILENT**; you turn it on when you need it.

RESET did not clear memory

It is not supposed to.

RESET is the bus's **RESET*** line. It does what pulling that line did: the processor goes to zero, boards return to their power-on state. RAM is RAM; it was not cleared on the real machine and it is not cleared here.

POWER is the only thing that loses memory. That is the difference between pressing a button and pulling a plug, and the manual keeps it.

macOS refuses to run it

"cannot be opened because the developer cannot be verified".

```
$ xattr -dr com.apple.quarantine ./swt pcsim
```

Once. It is not a comment on the program; it is what macOS does to every unsigned binary that arrives in a zip.

If that prints nothing, it worked — and if the flag was never there (a zip fetched with **curl** or **scp** is not marked; only one a browser or a mail client downloaded is), it also prints nothing and succeeds. That is the whole reason for **-dr** over a plain **-d**, which announces an error when there is nothing to remove.

Glossary

ACIA — Asynchronous Communications Interface Adapter. The Motorola 6850 chip at the heart of the SWTPC MP-S console and the Altair 680b's serial ports. It has two registers a program can see: a status register and a data register. Everything a program does with a serial port on this machine, it does through one of those.

ATTN — See **STOP**. The former name for the STOP key; the `CONSOLE attn=` spelling still works as an alias.

backplane — The PCB with the connectors on it that every board plugs into. It is not itself one of them: nothing in a machine file fits a backplane, because the backplane is what a machine file describes the contents of. On a SWTPC 6800 it is the **SS-50 bus** — the motherboard carrying the CPU and memory — with the **SS-30 I/O bus** hanging off it. The backplane is the machine.

board — Anything that plugs into the backplane: memory, a serial port, a disk controller, the processor itself. It is the word this program uses everywhere — `BOARDS` lists them, `[[board]]` fits them in a machine file — and a board is the thing you add, remove, `SHOW` and `SET`. See also **card**, which means the same thing.

card — The same object as a **board**. The period hardware and its manuals said "card", and this manual keeps the word where the sentence is about the physical thing somebody bought, socketed chips into and set jumpers on. Everywhere else it says board.

contention — Two boards answering the same address. On real hardware both would drive the data bus at once and you would get a byte that is neither of theirs, intermittently, in a way that would take you a week. `SHOW BUS CONTENTION` names them instead.

cycle — The unit of emulated time: one tick of the CPU's clock. Every 6800 instruction costs a known number of them, and that is how this program knows what time it is. At 1 MHz a cycle is one microsecond, and a cassette takes the time it takes because it costs a fixed number of them.

DC-4 — SWTPC's floppy-disk controller: a WD179x on the SS-30 bus, with up to four 5.25" drives. It is what FLEX boots from. `dc4` in a machine file.

decode — What a board does when it recognises an address as its own and answers. A board that does not decode an address stays silent and lets somebody else have it. Which board decodes which address is the entire question of how a machine is put together.

endpoint — In this program, the thing on the far end of a serial unit's cable: `console`, `null`, `loopback`, a TCP socket, or a real serial port. The serial chapter has the complete list; there are no others.

FLEX — TSC's disk operating system for the 6800, and the reason the DC-4 is here. It boots off a floppy to a `+++` prompt; `CAT` lists a disk, and files have names, extensions and a `SAVE`.

floating bus — What the data bus reads when nothing is driving it. Read an address no board decodes and you get whatever the bus floats to, not an error — that is the absence of a board, and it looks like a fixed byte.

front panel — The controls a real operator used to examine memory, deposit bytes, and start and stop the CPU. On these machines that job belongs to the ROM monitor — SWTBUG's and MON680's `M / G` commands — and to the `swtpcsim>` prompt, which does EXAMINE, DEPOSIT, RUN, STEP and RESET from the host with a much better view than a panel of switches gave.

FSK — Frequency Shift Keying. Encoding bits as two audible tones — one for a zero, one for a one. It is how the KCACR gets data onto a cassette, and it is why a loading tape sounds the way it does. The tones are **not** standard across machines: a board can only read the modulation its own modem was built for, so mounting a recording in the wrong one is refused rather than decoded. See **Kansas City standard** and the tapes chapter.

Kansas City standard — The 1975 agreement on how microcomputers should record data on ordinary audio cassettes, named for the meeting that settled it. A one is eight cycles of 2400 Hz, a zero is four cycles of 1200 Hz, and both take the same time — which is the whole point: a receiver counts cycles instead of trusting a clock, so a tape that plays 5% slow still reads. 300 baud. The Altair 680b's KCACR reads and writes it. See also **FSK**.

KCACR — The Altair 680b's Kansas City audio-cassette interface: a UART recording Kansas City FSK, memory-mapped at `F010 / F011`, with a loader/punch PROM. `680kcacr` in a machine file.

MCP — Model Context Protocol. The protocol an AI assistant uses to call structured tools. `swtpcsim --mcp` speaks it, so an assistant can drive the machine directly. See the MCP chapter.

MON680 — the Altair 680b's ROM monitor. Prompt `. ;` `M` examines and deposits a byte, `G` goes, `J` jumps to an address.

monitor — Two things wear the name here. A **ROM monitor** (SWTBUG, MON680) is the program burned into the machine's ROM that examines memory and runs code — the software a bare 6800 was shipped with. The `swtpcsim>` **monitor** is the host prompt you reach with `^E`: not a program running inside the machine, but you, standing in front of it, with a much better console than the hardware ever had.

PROM — Programmable Read-Only Memory. A chip with a program burned into it that survives power-off. A 6800's ROM monitor lives in one, which is the only reason the machine can start at all: something has to already be there to talk to.

sector — The smallest chunk of a disk you can read or write. FLEX disks use 256-byte sectors.

SS-50 / SS-30 bus — SWTPC's two buses. The **SS-50** carries the CPU, memory and the system signals — the motherboard everything plugs into. The **SS-30** is the I/O bus that hangs off it, an eight-slot window at

`$8000 – $801F` where the MP-S console and the DC-4 floppy controller live; slot N sits at `$8000 + 4 · N`.

STOP — The STOP control, and the key that presses it: `^E` by default. It stops the running CPU and returns you to the monitor without disturbing the machine — not RESET, not POWER — so a bare `RUN` resumes at the instruction it was about to execute. The host intercepts it before the guest sees the byte, so no guest program can take it from you. Move it with `CONSOLE stop=` (the older `attn=` still works).

SWTBUG — the SWTPC 6800's ROM monitor, in a 2716 at `$E000`. Prompt `$`; `D` boots a disk, `M` examines memory, `G` goes.

TDRE — Transmit Data Register Empty. The bit in the 6850's status register that means "the board has room for another character". A program that wants to print polls it until it sets. A serial port connected to `null` sets it forever, which is why writing to nothing works fine.

track — One concentric ring of sectors on a disk. The head steps in and out to reach a track and does not move again to reach the sectors on it.

UART — Universal Asynchronous Receiver/Transmitter. The chip that turns a byte into a sequence of bits on a wire and back again. The ACIA is one.

unit — One channel on a board that moves characters — the socket on the back. The MP-S console has one (`tty`); the 680b Universal I/O board adds a second serial line (`serial`). `CONNECT uio0:serial` names the board and then the unit.

Boards and their parameters

Every key below is a key you may write in a machine file, and the *same* key you may `SET` at the monitor prompt. That is not a coincidence and it is not a convention: a board's properties **are** its TOML schema, so there is nothing here that could disagree with the program.

Numbers follow the one rule: **on the wire → hex, never on the wire → decimal**. A port is hex; a baud rate and a drive count are decimal. The defaults below are printed in each property's own base.

The catalogue is **grouped by function** — CPU, memory, disk, serial, and so on — and within a group the boards are in **alphabetical order**.

CPU

Type	What it is
<code>6800</code>	Altair 680b / SWTPC CPU board: a Motorola 6800
<code>6809</code>	CPU board: a Motorola 6809

Memory

Type	What it is
<code>memory</code>	RAM/ROM board: plain, unbanked memory regions

Disk

Type	What it is
<code>dc4</code>	SWTPC DC-4: WD179x 5.25" floppy controller

Serial

Type	What it is
<code>680io</code>	Altair 680b onboard I/O: 6850 console and strap port
<code>mps</code>	SWTPC MP-S: one 6850 serial port on an SS-30 slot

Tape

Type	What it is
<code>680kcacr</code>	Altair 680b KCACR: Kansas City audio cassette

Parallel and printer

Type	What it is
<code>680uio</code>	Altair 680b Universal I/O: 6850 serial port and 6820 PIA

CPU

6800

Altair 680b / SWTPC CPU board: a Motorola 6800. Decodes nothing -- it drives the bus. Memory-mapped I/O

Units: `6800` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
<code>clock_hz</code>	int	<code>0</code>	<code>0 .. 100000000</code>	Crystal on the board. 0 runs flat out -- as fast as the host can.
<code>idle</code>	bool	<code>true</code>	<code>on off</code>	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
<code>achieved_hz</code>	int	—	—	LIVE: cycles per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. (read-only — not a key you may set)

6809

CPU board: a Motorola MC6809. It decodes nothing. It drives the bus and takes IRQ from the bus. Its FIRQ input is not connected

Units: `6809` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
<code>clock_hz</code>	int	0	0 .. 100000000	Crystal on the board. 0 runs flat out -- as fast as the host can.
<code>idle</code>	bool	true	on off	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
<code>achieved_hz</code>	int	—	—	LIVE: cycles per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. (read-only — not a key you may set)

Memory

memory

RAM/ROM board: a list of regions -- plain, unbanked memory

`[[board.region]]` — a list you may add

Key	Kind	Legal	Meaning
<code>type</code>	enum	ram rom	RAM, or ROM (which needs a <code>mount</code> , unless you want an empty socket)
<code>at</code>	int	0x0 .. 0xFFFF	Where it starts. An address: 0000, F800
<code>size</code>	int	1 .. 65536	How much. Decimal, and it takes a suffix: 48K, 1024, 2M
<code>mount</code>	string	text	The ROM image. A file (relative to THIS FILE), or builtin:
<code>relocate</code>	bool	on off	Move the image so its first record lands at <code>at</code> (like LOAD's AT), rather than requiring it to self-place there -- for a mirror-decoded ROM

Board properties

Key	Kind	Default	Legal	Meaning
<code>fill</code>	enum	<code>random</code>	<code>zero</code> <code>random</code>	RAM contents at power-on: zero random (real RAM is not zeroed)
<code>seed</code>	int	<code>1</code>	any	Seed for fill=random. The same seed fills RAM the same way at every POWER, so a run is repeatable; change it for a different junk pattern
<code>pages</code>	string	—	—	the composite page map -- which pages this board answers for. Derived from the regions you declared (read-only — not a key you may set)

Disk

`dc4`

SWTPC DC-4 floppy disk controller: a WD179x (1 MHz) with up to four 5.25" drives (drive0..3). Spans two SS-30 ports -- a true-sense drive/side select latch at \$8014 and the WD179x registers at 8018–801B (default). Memory-mapped, DRQ-pollled. FLEX 2.0/3.0 boots from it via SWTBUG's 'D' command

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

[[board.drive]] — a list you may add

Key	Kind	Legal	Meaning
<code>unit</code>	int	<code>0 .. 3</code>	Which drive (0..3)
<code>mount</code>	string	text	The disk image to put in it. Relative to THIS FILE.
<code>readonly</code>	bool	<code>on</code> <code>off</code>	Write-protect the disk. The drive senses it, so the guest is never told (<i>also writeprotect</i>)
<code>media</code>	enum	<code>flex35</code> <code>flex40</code> <code>flexds</code> <code>cp68</code>	Force the format instead of probing the image's size

Board properties

Key	Kind	Default	Legal	Meaning
base	int	0x8018	0x8008 .. 0x801C	WD179x block base (\$8018 = SS-30 slot 6); registers at base..base+3, drive-select latch four below (base-4 = \$8014)
speed	enum	full	full real	Drive timing. full: collapse seek and byte timing so the disk keeps up with the simulator at whatever speed it runs (the default -- FLEX polls DRQ/BUSY and cannot tell the difference). real: model the WD179x's actual timing -- 30ms/step seeks and per-byte data rate, and the Lost Data that a too-slow driver would hit
drives	int	4	1 .. 4	Drives on the controller (binary select D0..D1, 0-3)

Serial

680io

Altair 680b onboard I/O: a 6850 ACIA console ('tty') at F000/F001 and the config-strap read port at F002. Memory-mapped

Units: tty (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
straps	int	0x0	0x0 .. 0xFF	Config straps read at F002: bit7 No-Terminal (0=terminal), bit2 stop bits

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
<code>baud</code>	int	<code>9600</code>	<code>50 .. 76800</code>	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
<code>dcd</code>	enum	<code>ground</code>	<code>ground</code> <code>wired</code>	/DCD pin: grounded on the card, or wired to the connector
<code>cts</code>	enum	<code>ground</code>	<code>ground</code> <code>wired</code>	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
<code>lines</code>	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK (read-only — not a key you may set)
<code>connect</code>	string	<code>null</code>	text	The endpoint on the other end of the line (CONNECT sets this)

`mps`

SWTPC MP-S serial interface: a 6850 ACIA console ('tty') on an SS-30 slot, control/status at the slot base and Rx/Tx data at base+1 (default 8004/8005, the console slot). Memory-mapped; the ACIA IRQ pulls the 6800 IRQ. The board SWTBUG/MIKBUG's terminal routines assume

Units: `tty` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
<code>base</code>	int	<code>0x8004</code>	<code>0x8000 .. 0x801C</code>	SS-30 slot base ($\$8000 + \text{slot} * 4$); control/status at base, Rx/Tx at base+1

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
<code>baud</code>	int	9600	50 .. 76800	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
<code>dcd</code>	enum	ground	ground wired	/DCD pin: grounded on the card, or wired to the connector
<code>cts</code>	enum	ground	ground wired	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
<code>lines</code>	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK (read-only — not a key you may set)
<code>connect</code>	string	null	text	The endpoint on the other end of the line (CONNECT sets this)

Tape

680kcacr

Altair 680b KCACR audio-cassette interface: a 1602-family UART recording Kansas City Standard FSK, memory-mapped at F010 (status/control) and F011 (data), active-LOW. Adds software motor control (control D7=on, D6=off) and interrupt-driven transfer (D0/D1 enables pull the 6800 IRQ). MOUNT a tape, WIND/REWIND it

Units: `tape` (tape, MOUNT)

Board properties

Key	Kind	Default	Legal	Meaning
<code>baud</code>	int	300	50 .. 25000	Line rate. A JUMPER on the real card -- software cannot change it
<code>data_bits</code>	int	8	5 .. 8	Data bits per character. The NDB1/NDB2 pads
<code>stop_bits</code>	int	2	1 .. 2	Stop bits. The NSB pad: GND = 1, +V = 2
<code>parity</code>	enum	none	none odd even	The NPB/POE pads: none odd even
<code>motor</code>	enum	—	—	Tape-recorder motor relay (guest-driven: STA F010 7F = on, BF = off) (read-only — not a key you may set)

Unit `tape` — `[board.unit.tape]`

Key	Kind	Default	Legal	Meaning
<code>mode</code>	enum	<code>play</code>	<code>play</code> <code>record</code>	Which way the bytes go: play loads from the file, record saves to it
<code>format</code>	enum	<code>auto</code>	<code>auto</code> <code>raw</code> <code>kcs300</code>	How to read the mounted file: auto raw fsk300
<code>leader</code>	int	<code>15</code>	<code>0</code> .. <code>120</code>	Seconds of idle tone before recorded data, when writing audio
<code>trailer</code>	int	<code>5</code>	<code>0</code> .. <code>120</code>	Seconds of idle tone after recorded data, when writing audio
<code>waveform</code>	enum	<code>square</code>	<code>square</code> <code>sine</code>	Carrier shape when writing audio: square (like real hardware) sine
<code>level</code>	int	<code>36</code>	<code>1</code> .. <code>100</code>	Recording level as a percent of full scale, when writing audio
<code>rate</code>	enum	<code>full</code>	<code>full</code> <code>real</code>	Playback speed: full (as fast as the guest reads) real (wall-clock baud)
<code>detected</code>	string	—	—	What the mounted tape turned out to be (empty if nothing is mounted) (read-only — not a key you may set)
<code>position</code>	string	—	—	Where the tape head is now: mm:ss / total (percent) -- read-only (read-only — not a key you may set)
<code>counter</code>	enum	<code>on</code>	<code>on</code> <code>off</code>	Live tape counter on the console during a load: on off
<code>stop</code>	string	<code>off</code>	<code>off</code> <code>end</code> <code>mm:ss</code>	Auto-stop playback at this time: off end <u>mm:ss</u>

Parallel and printer

`680uio`

Altair 680b Universal I/O: a second 6850 ACIA serial port ('serial') and a 6820 PIA parallel port (sections 'p1a/p1b', 'p2a/p2b' with `pias=2`) in an S9-relocatable window (default base F000: serial F006/F007, PIA F008-F00F), plus fixed switch inputs at F003 and a non-latched output at F010-F013. Memory-mapped, active-high

Units: `serial` (serial, CONNECT), `p1a` (serial, CONNECT), `p1b` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
<code>base</code>	int	<code>0xF000</code>	<code>0xF000 .. 0xF0F0</code>	S9 window base (F000 + position*0x10); serial at base+6, PIAs base+8..+F
<code>pias</code>	int	<code>1</code>	<code>1 .. 2</code>	6820 PIAs populated: 1 (PIA-C only) or 2 (PIA-C + PIA-B)
<code>sense</code>	int	<code>0x0</code>	<code>0x0 .. 0xFF</code>	Switch inputs read at F003 (fixed, read-only tri-state)
<code>nlout</code>	bool	<code>true</code>	<code>on off</code>	Decode the F010-F013 non-latched output (off = IC A1 removed, KCACR owns F010/F011)

Unit `serial` — `[board.unit.serial]`

Key	Kind	Default	Legal	Meaning
<code>baud</code>	int	<code>9600</code>	<code>50 .. 76800</code>	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
<code>dcd</code>	enum	<code>ground</code>	<code>ground wired</code>	/DCD pin: grounded on the card, or wired to the connector
<code>cts</code>	enum	<code>ground</code>	<code>ground wired</code>	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
<code>lines</code>	string	<code>—</code>	<code>—</code>	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK (read-only — not a key you may set)
<code>connect</code>	string	<code>null</code>	text	The endpoint on the other end of the line (CONNECT sets this)

Unit `p1a` — `[board.unit.p1a]`

Key	Kind	Default	Legal	Meaning
<code>connect</code>	string	<code>null</code>	text	The endpoint on the other end of this PIA section (CONNECT sets this)

Unit `p1b` — `[board.unit.p1b]`

Key	Kind	Default	Legal	Meaning
<code>connect</code>	string	<code>null</code>	text	The endpoint on the other end of this PIA section (CONNECT sets this)

The built-in machines

A built-in is a machine file that lives **inside the binary** — the same format you would write yourself, with nothing special about it. Name one and you get it in any directory on earth:

```
$ swtpcsim swtpc
```

`swtpcsim --list` prints this table, and `swtpcsim -x 'SHOW MACHINE' <name>` shows what is actually in one.

Machine	What it is
<code>altair680</code>	The Altair 680b -- MITS's second machine, and a different animal from the 8800.
<code>swtpc</code>	The SWTPC 6800 -- Southwest Technical Products' 1975 computer, the machine this simulator is named for.